MICRO PROJECT REPORT

FULL STACK WEB DEVELOPMENT

SEATSURE: CONCURRENCY-SAFE CAMPUS EVENT REGISTRATION WITH AUTO-PROMOTING WAITLISTS

Submitted by:
Bhavya Sharma
2511021060291

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING (AI AND FUTURE TECHNOLOGIES)

Under the guidance of
Mr. Vaibhav Giri



Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Full Stack Web Development** for the micro project titled "SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists" is submitted by Bhavya Sharma (University Registration No. 2511021060291) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (Computer Science and Engineering – AI and Future Technologies) of Alliance University. It is a bonafide record of work carried out under my supervision and guidance during the academic year 2026–27. The work presented in this report is original and has not formed the basis for the award of any other degree to the candidate or to anybody else.

……

Mr. Vaibhav Giri



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists** (Full Stack Web Development), submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (AI and Future Technologies)** at **Alliance University**, is a record of project work carried out under the supervision of **Mr. Vaibhav Giri**.

The design, implementation and test results described in this report belong to the SeatSure project submitted with it. The course material, documentation and open-source libraries that the work builds on are acknowledged in the references.

This report has not been submitted, in part or in full, for any other degree or award.

Name of the Student	University Registration Number	Signature
Bhavya Sharma	2511021060291	

Acknowledgement

I would like to express my sincere gratitude to everyone who helped me complete this micro project, **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists**.

First, I thank my guide, **Mr. Vaibhav Giri**, for the guidance, feedback and encouragement provided throughout the project. I am also grateful to the mentors at Kalvium, whose Full Stack Web Development livebook sessions provided the concepts on which this work is built.

I also thank my peers in my Kalvium squad for the discussions and feedback that helped shape the project.

Working on SeatSure taught me that a feature is finished only when it has been shown to work under realistic conditions, with automated tests, measurements and failure scenarios, and not merely when it works once on my own screen.

Finally, I thank my family and friends for their support and patience during the semester.

Thank you.

Bhavya Sharma

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I – Introduction **7**

1.1 Problem Statement 7
1.2 Objectives 8
1.3 Scope of this Report 8
1.4 Theory 8

Chapter II – System Configuration **13**

2.1 System Architecture 13
2.2 Algorithm 15
2.3 Source Code 16
2.4 Connection with Back-End Web Development and Advanced Database Systems 30
2.5 Setting Up and Running the Project 31
2.6 CRUD over REST: Route Mapping and Validation 32

Chapter III – Results **34**

3.1 Testing Approach 34
3.2 Test Results Summary 34
3.3 Results 35
3.4 Live API Test Cases 41
3.5 Discussion 42

Chapter IV – Conclusion **43**

4.1 Limitations 43
4.2 Future Scope 43

References **45**

Appendix A – Test Cases **46**

Appendix B – Concept Traceability **50**

Abstract

Student clubs at Alliance University run workshops, hackathons and talks in rooms with a fixed number of seats, yet registrations are still collected through Google Forms and WhatsApp. These tools cannot enforce a seat limit, keep a fair waitlist or tell organisers how well their events fill. This report describes the Full Stack Web Development part of **SeatSure**, an integrated micro project that replaces them with a web application in which a seat limit can never be exceeded, a cancelled seat passes automatically to the next person on the waitlist, and organisers see live analytics.

The client is a React 19 single-page application built with Vite. It applies the concepts of Modules 1–3 of the course: Context and custom hooks for shared state; React Hook Form for validated forms whose fields are generated from each event's own questions; `useReducer` for the catalogue filters; `React.memo` verified by a render-count test; route-level code splitting with `React.lazy`; error boundaries; and nested layout routes. Server data flows through one Axios instance with interceptors and is cached by TanStack Query, which provides cursor pagination with infinite scroll, mutations with selective cache invalidation, an optimistic cancellation with rollback, and polling for live seat counts. A personal to-do list ("My tasks") shows create, read, update and delete over REST in their simplest form. The client talks to an Express API backed by PostgreSQL through Prisma ORM.

All 32 automated client tests pass (Vitest and React Testing Library). They show, for example, that a cancelled pass moves before the server answers and moves back when the server refuses, that three keystrokes in the search box produce a single request, and that a memoised list re-renders only the card whose data changed. Code splitting keeps a student's first download at 126.7 KB of gzipped JavaScript, while the organiser area adds 23.7 KB only when it is opened. The report concludes that a carefully layered React client can feel fast and forgiving while the server remains the single source of truth.

Keywords: React, TanStack Query, React Hook Form, optimistic updates, cursor pagination, code splitting, error boundaries, PERN stack

Chapter I – Introduction

A full-stack application is judged by what happens when a person clicks a button. The click has to travel from a React component, across the network, through an API and into a database, and the answer has to come back and update the screen without confusing the person who clicked. SeatSure is built around one such click, "Register", which must stay correct even when fifty students press it at the same moment for the last few seats of a popular workshop.

SeatSure has two kinds of user. **Students** browse upcoming events, register (or join a waitlist when an event is full), cancel, and follow their passes and notifications. **Organisers** of a club create events with their own registration questions, publish them, check attendees in, export attendee lists and read analytics about how their events fill. Every user also keeps a personal to-do list, **My tasks**, with full create, view, update (edit or tick done) and delete operations.

The project is one application split into three folders that match the three subjects of the integrated project: `/frontend` (Full Stack Web Development: the React interface), `/backend` (Back-End Web Development: the Express API) and `/db` (Advanced Database Systems: the PostgreSQL schema, the Prisma data-access layer and MongoDB). Each subject has its own report, and each report is written to be read on its own. This report covers **Full Stack Web Development**, whose code lives mainly in `/frontend`, and explains how it connects to the other two parts.

1.1 Problem Statement

Student clubs at Alliance University run workshops, hackathons and talks in venues with a fixed number of seats, but registrations are collected through Google Forms and WhatsApp. These tools cannot enforce a seat limit, so popular events get overbooked when many students register at once; waitlists are tracked by hand; seats freed by cancellations are not offered to the next person; and organisers have no data on fill rates or attendance.

SeatSure is a web application where students browse events and register with a hard seat cap that stays correct under simultaneous sign-ups, cancellations automatically promote the next waitlisted student, and organisers manage attendees and view registration analytics.

For the user interface this creates three concrete challenges. First, the number of free seats on screen must stay close to the truth while other students register, without flooding the server with requests. Second, a student must get immediate, clear feedback (a confirmed seat, a

waitlist position or an understandable error) even though the final decision is made on the server. Third, the organiser tools (forms with variable questions, paginated attendee lists and analytics) must not slow down the experience of the much larger group of students who never use them.

1.2 Objectives

- Build a responsive React interface for students and organisers that works on a 390-pixel phone screen as well as on a laptop.
- Keep server data in one client cache (TanStack Query) with freshness rules chosen per kind of data, and refresh only what a change affects.
- Make registration and cancellation feel immediate while the server remains the single source of truth: optimistic updates with rollback, and server validation errors shown on the right form field.
- Load long lists in pages: cursor pagination with infinite scroll for the catalogue, and offset pagination for organiser tables.
- Contain failures: error boundaries around risky parts of the page and one predictable error shape from the API client.
- Prove each behaviour with an automated test.

1.3 Scope of this Report

This report covers Modules 1–3 of the Full Stack Web Development course: advanced React (Module 1), frontend–backend integration (Module 2) and full stack with a database (Module 3). Lesson numbers such as **M2.8** refer to the lessons of the course livebook. Real authentication with sessions or JSON Web Tokens is taught in Module 4, so SeatSure identifies people through a demo user switcher that sends an `X-User-Id` header; the client already handles a 401 response the way a real login flow would. Containers and deployment (Module 5) are also outside the scope.

1.4 Theory

Table 1.1 lists every concept applied in the client and where it is used. The paragraphs after the table explain the theory behind each group in plain terms; Chapter II shows the code.

Table 1.1: Full Stack concepts applied in SeatSure (Modules 1–3)

Lesson	Concept	Used in SeatSure for
M1.4	Context API	The current user and toast messages, available to every component without prop drilling
M1.5	Custom hooks	`useCurrentUser`, a debounced search value, an infinite-scroll sentinel
M1.6	React Hook Form	Registration form generated from each event's questions; event editor with `useFieldArray` and cross-field rules; the task form
M1.7	`React.memo`, `useMemo`, `useCallback`	Catalogue cards that skip re-rendering when a poll returns unchanged data (measured)
M1.8	Code splitting (`React.lazy` + `Suspense`)	Every page is a separate chunk; the organiser area downloads only when opened
M1.9	Error boundaries	App-level boundary that resets on navigation; one boundary per analytics panel
M1.10	Nested and layout routes	`AppLayout` and `OrganiserLayout` render shared chrome around an `<Outlet/>`
M1.11	`useReducer`	Five related catalogue filters changed through named actions
M1.12	Compound components	`<Card>` with `Card.Header`, `Card.Body` and `Card.Footer`, used by every card
M1.13	Feature-based architecture	`features/`, `shared/` and `services/` folders with barrel files
M2.1	CORS, environment variables, two servers	CORS allow-list on the API, `VITE_API_URL`, one command starts both servers
M2.2	Axios instance	One configured `apiClient`; components never call Axios directly
M2.3–2.4	TanStack Query, caching, `staleTime`	Shared `QueryClient` defaults; freshness chosen per kind of data
M2.5	Query parameters and filtering	Filter state → query key → Axios `params`
M2.6–2.7	Mutations, selective invalidation	Register; create and edit events; add, edit and delete tasks; check-in. Only affected caches refetch
M2.8	Optimistic updates	Cancelling a pass, or ticking a task done, changes instantly and rolls back on failure
M2.9	React Hook Form + mutation	Server 400/422 field errors shown under the matching input
M2.10	API error handling	Interceptors, one `ApiError` shape, 401 redirect, retry only when it can help
M2.11	Custom API hooks, key factory	`useEvents.js`, `useMyRegistrations.js`, `queryKeys.js`
M3.1–3.4	Prisma behind Express; `include`/`select`/`_count`; Prisma errors	Event detail with waitlist count; "already registered" message from a unique-index error
M3.5	Nested data in React	"My passes": registration → event → club and venue
M3.7	Filtering and sorting end to end	Catalogue filters reach Prisma `where` and `orderBy`
M3.8	Offset pagination	Organiser dashboard and attendee list (`page`, `pageSize`, `totalPages`)
M3.9	Cursor pagination	Catalogue pages with `useInfiniteQuery` and `nextCursor`
M3.11	Infinite scroll	`IntersectionObserver` on a sentinel below the list
M3.12	Data freshness	Polling every 5 s (event), 15 s (notifications), 30 s (catalogue); refetch on focus

1.4.1 Advanced React (Module 1)

Context and custom hooks (M1.4, M1.5). Some values are needed almost everywhere. Who the current user is, for example, decides what the header shows, which passes are listed and whether the organiser tools appear. Passing such a value through every level of the component tree ("prop drilling") couples components that have nothing to do with it. React Context makes the value available to any component below a provider. Because every consumer re-renders when the value changes, a context should hold little and change rarely. A custom hook such as `useCurrentUser` wraps `useContext`, so components do not depend on how the value is stored, and it fails loudly if a component is rendered outside the provider. Custom hooks also package behaviour rather than data, such as a debounced value or an intersection observer.

Forms (M1.6). React Hook Form registers inputs with `register` and validates them on submit or on blur, without storing every keystroke in React state, which keeps re-renders low. Rules are declared next to each field (`required`, `min`, `max` and custom `validate` functions) and the `errors` object holds one message per field. `useFieldArray` manages a growing list of repeated field groups, such as an organiser's custom questions.

Rendering performance (M1.7). React re-renders a component whenever its parent re-renders, even when its props are unchanged. `React.memo` skips that render when the props are shallowly equal, `useMemo` keeps an object's identity stable between renders and `useCallback` does the same for functions. These tools have their own cost, so the rule taught in the course is to measure first and memoise only where wasted renders have been shown.

Code splitting, error boundaries and nested routes (M1.8–M1.10). `React.lazy` turns a page into a separate JavaScript chunk that the browser downloads the first time the page is needed; `<Suspense>` shows a fallback meanwhile. An error boundary catches errors thrown while rendering its children and shows a fallback instead of unmounting the whole application; the `react-error-boundary` package adds a reset function and reset keys. React Router v6 nests routes: a layout route renders the shared chrome and an `<Outlet/>` in which the matched child route appears.

State and composition (M1.11–M1.13). When several pieces of state change together, `useReducer` keeps every transition in one pure function with named actions, which is easier to reason about and to test than several separate setters. Compound components (`Card`, `Card.Header`, `Card.Body`) let callers compose a layout instead of configuring one component through many props. Grouping code by feature, with a barrel file as each feature's single public

entry point and a `services/` layer for all HTTP calls, keeps features independent of one another.

1.4.2 Frontend–Backend Integration (Module 2)

Connecting two servers (M2.1, M2.2). In development the React app (Vite, port 5173) and the API (Express, port 4000) run on different origins, so the browser blocks API calls unless the server allows the client's origin through CORS. The API address comes from the environment variable `VITE_API_URL`, and every request goes through one configured Axios instance, so the base URL, timeout and default headers are defined once.

Server state with TanStack Query (M2.3–M2.5). Data that lives on the server is only a copy on the client: it can be out of date, and several components may need it at the same time. TanStack Query stores each response under a *query key*, shares it between components and refetches it in the background according to `staleTime`, `refetchInterval` and window-focus rules. When the filters are part of the query key, changing a filter produces a new key and therefore a new request, while the earlier result stays in the cache.

Changing data (M2.6–M2.9). A mutation sends a POST, PATCH or DELETE request. After it succeeds, the client *invalidates* only the queries whose data the change affected, so that they refetch. An *optimistic update* goes one step further: the cache is changed before the server answers, a snapshot is kept, and the snapshot is restored if the server rejects the change. Forms submit through mutations, and the server's field errors are displayed under the matching inputs.

Errors and structure (M2.10, M2.11). Interceptors run on every request and response, which makes them the right place to attach headers, normalise errors and react to a 401. Query retries should be limited to failures that a retry can actually fix. Queries and mutations are wrapped in custom hooks per feature, and every cache key comes from a single key factory so that invalidation is precise.

1.4.3 Full Stack with a Database (Module 3)

Prisma behind Express (M3.1–M3.4). The Express routes read and write PostgreSQL through Prisma ORM. `include`, `select` and `_count` shape a response, including related rows and counts, in a single query. Prisma errors such as P2002 (unique constraint violated) and P2025 (record not found) are translated by the server's error middleware into HTTP status codes and messages that React can display.

Rendering nested data (M3.5). A nested `include` response, such as a registration with its event, club and venue, must render safely while it is loading and when an optional relation is missing. Optional chaining and default values prevent “cannot read property of undefined” crashes.

Lists that grow (M3.7–M3.11). Filtering and sorting belong in the database: the filter state travels from React through query parameters to Prisma’s `where` and `orderBy`. *Offset pagination* (`skip`/`take` plus a total count) suits numbered pages. *Cursor pagination* returns a `nextCursor`, the position of the last row, instead; it stays stable when new rows are inserted, which suits a feed, and `useInfiniteQuery` consumes it page by page. Infinite scroll calls `fetchNextPage` when an `IntersectionObserver` reports that a sentinel element below the list has become visible.

Freshness (M3.12). Polling with `refetchInterval`, refetching when the window regains focus, and `staleTime` trade freshness against server load. Each kind of data gets its own setting: seat counts on an open event page change quickly; the list of clubs almost never changes.

Chapter II – System Configuration

SeatSure was developed and tested on a single laptop. Table 2.1 lists the hardware and the software versions. The client libraries are the ones the course uses; the server and databases are listed because the client was always tested against them.

Table 2.1: Hardware and software configuration

Component	Configuration
Processor and memory	Apple M4 (10 cores), 16 GB RAM
Operating system	macOS 26.7 (arm64)
Runtime	Node.js 22.23.2, npm 10.9.8 (npm workspaces: `frontend`, `backend`, `db`)
Client build tool	Vite 8.3.5 (development server on port 5173, production build with code splitting)
UI library and routing	React 19.3.1, React DOM 19.3.1, React Router DOM 6.30.6
Server state and HTTP	TanStack Query (React Query) 5.104.5, Axios 1.20.0
Forms and error handling	React Hook Form 7.91.0, react-error-boundary 6.1.6
API consumed	Express 5.3.0 REST API under `/api/v1` (port 4000); both servers start with `npm run dev` (concurrently 10.0.6)
Databases behind the API	PostgreSQL 18.4 through Prisma ORM 7.10.0; MongoDB 8.2 through Mongoose 9.11.4
Testing	Vitest 5.0.5, React Testing Library 16.3.3, jsdom 27.4.0
Browser and tools	Google Chrome 156 with DevTools (Network tab); Git for version control

2.1 System Architecture

Figure 2.1 shows the three parts of SeatSure and how they connect. The client never talks to a database directly: it calls the REST API through Axios. The API applies the business rules and owns every transaction. PostgreSQL is the single source of truth; MongoDB holds derived data (notifications and an activity feed) that is filled asynchronously from PostgreSQL.

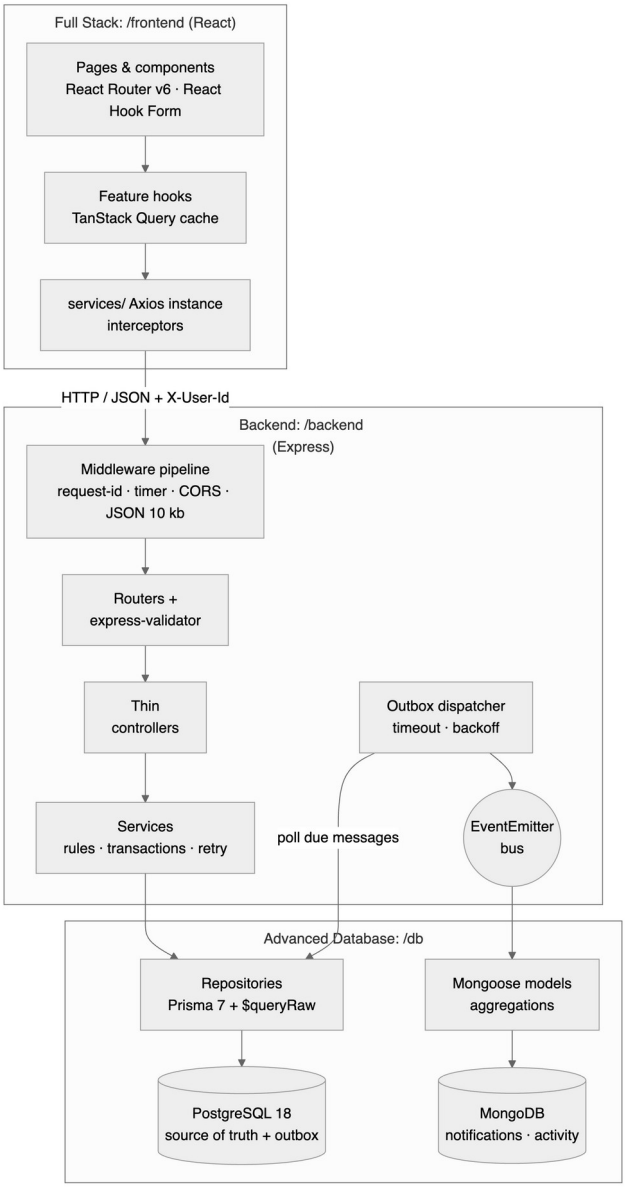


Figure 2.1: SeatSure architecture: client, API and data layers

Inside `/frontend`, code is grouped by feature (Listing 2.1). Each feature folder holds its pages, components and hooks, and exposes a barrel file (`index.js`) as its only public entry point. Code that several features share lives in `shared/`, and every HTTP call goes through `services/` (M1.13).

Listing 2.1: Feature-based folder structure of `frontend/src` (abridged)

```
frontend/src/
├── app/            router.jsx (lazy, nested routes) · queryClient.js (cache
    defaults)
├── layouts/        AppLayout.jsx (header, error boundary, Suspense, <Outlet/>)
├── services/       apiClient.js (Axios instance + interceptors), eventsApi.js, …
├── features/
│   ├── events/         EventsPage, EventCard, EventFilters, RegistrationPanel,
│   │                   hooks/useEvents.js, state/filtersReducer.js, index.js
│   ├── registrations/  MyPassesPage, PassCard, hooks/useMyRegistrations.js
│   ├── notifications/  NotificationBell, NotificationsPage,
    hooks/useNotifications.js
│   ├── organiser/      OrganiserLayout, OrganiserHome, EventFormPage, AttendeesPage,
│   │                   AnalyticsPage, hooks/useOrganiser.js
│   └── users/          CurrentUserContext, UserSwitcher, ChooseUserPage, index.js
└── shared/         components/ (Card, Field, Pagination, Toast, …), hooks/,
    lib/queryKeys.js
```

2.2 Algorithm

The three most important behaviours of the client are written below as step-by-step algorithms. The lesson that each step applies is given in brackets.

Algorithm 1: Browsing the catalogue (filters, cursor pagination, infinite scroll)

1. The student changes a filter. `filtersReducer` returns the new filter state (M1.11); search text is debounced for 300 ms (M1.5).
2. `useMemo` builds a parameter object from the non-empty filters. It becomes part of the query key `['events', 'list', params]` (M2.5, M1.7).
3. `useInfiniteQuery` requests the first page, `GET /api/v1/events?…&limit=12`. The API returns 12 events and `meta.nextCursor`.
4. The page renders one card per event and places an invisible sentinel element below the grid.
5. When the sentinel scrolls into view, the `IntersectionObserver` calls `fetchNextPage`, which sends `cursor = nextCursor` (M3.9, M3.11).
6. Steps 4–5 repeat until `nextCursor` is `null`; the page then shows “You have reached the end”.
7. Every 30 s the list refetches in the background. Unchanged events keep their object identity, so memoised cards do not re-render (M1.7, M3.12).

Algorithm 2: Registering for an event

1. The event page loads the event (polled every 5 s) and builds one form field for each question stored in the event’s JSONB `questions` column (M1.6).

2. On submit, React Hook Form checks the rules locally. If any field fails, no request is sent.
3. The `useRegister` mutation sends `POST /api/v1/events/:id/registrations` with the answers (M2.6).
4. On 201 Created, a toast shows either "Seat confirmed" or the waitlist position, and the event, the catalogue lists, "My passes" and the activity feed are invalidated (M2.7).
5. On 422 with field errors, each error is placed under its field with `setError` (M2.9); any other error is shown as a toast.

Algorithm 3: Cancelling a registration optimistically

1. `onMutate` cancels in-flight refetches of "My passes", keeps a snapshot of the cached list and marks the pass as CANCELLED in the cache (M2.8).
2. The pass moves to the "Cancelled" section immediately.
3. The DELETE request runs. On success, a toast says whether the seat went to the next student on the waitlist.
4. On failure (for example 409, because the event has already started), `onError` restores the snapshot and a toast explains why.
5. `onSettled` invalidates "My passes", the event and the catalogue, so the cache matches the server in both cases.

2.3 Source Code

This section shows how and where each concept is implemented. Every listing is copied from the project, with its file path and line numbers; lines marked "omitted" were left out to keep the listing short. Comments in the code carry tags such as `[FULLSTACK M2.8]`, so running `grep -rn "FULLSTACK M" frontend/src` lists every place a concept is used. Appendix B maps each concept to its files and tests.

2.3.1 Routing, code splitting and layouts (M1.8, M1.9, M1.10)

Every page is declared with `React.lazy`, so Vite emits it as a separate chunk. The routes form a tree: `AppLayout` is the parent of all pages, and the organiser area is a second layout route nested inside it, with an index route and four children (Listing 2.2).

Listing 2.2: Lazy pages and nested layout routes (`frontend/src/app/router.jsx`, lines 10–47)

```
10  const EventsPage = lazy(() => import('../features/events/EventsPage.jsx'));
11  const EventDetailPage = lazy(() =>
        import('../features/events/EventDetailPage.jsx'));
    // … 2 lines omitted …
14  const TasksPage = lazy(() => import('../features/tasks/TasksPage.jsx'));
15  const OrganiserLayout = lazy(() =>
        import('../features/organiser/OrganiserLayout.jsx'));
    // … 5 lines omitted …
21
22  export const routes = [
23    {
24      element: <AppLayout />,
25      children: [
26        { index: true, element: <Navigate to="/events" replace /> },
27        { path: 'events', element: <EventsPage /> },
28        { path: 'events/:eventId', element: <EventDetailPage /> },
          // … 4 lines omitted …
33        {
34          path: 'organiser',
35          element: <OrganiserLayout />,
36          children: [
37            { index: true, element: <OrganiserHome /> },
38            { path: 'events/new', element: <EventFormPage /> },
              // … 2 lines omitted …
41            { path: 'analytics', element: <AnalyticsPage /> },
42          ],
43        },
44        { path: '*', element: <NotFoundPage /> },
45      ],
46    },
47  ];
```

The layout route renders the shared header and places the current page in its `<Outlet/>`. The outlet is wrapped in an error boundary whose `resetKeys` contain the current path, so a crash on one page never removes the header and disappears as soon as the user navigates elsewhere. The `<Suspense>` fallback is shown while a lazy chunk downloads (Listing 2.3).

Listing 2.3: Error boundary and Suspense around the outlet (`frontend/src/layouts/AppLayout.jsx`, lines 33–40)

```
33  <main className="main">
34    <ErrorBoundary FallbackComponent={ErrorFallback}
          resetKeys={[location.pathname]}>
35      {/* [FULLSTACK M1.8] Lazy pages show this fallback while their chunk
           downloads. */}
36      <Suspense fallback={<Spinner label="Loading page" />}>
37        <Outlet />
38      </Suspense>
39    </ErrorBoundary>
40  </main>
```

The effect is measurable with `npm run report:bundle`, which reads Vite's build manifest. The production build contains 26 JavaScript files. A student who opens the catalogue downloads 11 of them (126.7 KB gzipped); the organiser area's ten chunks (23.7 KB) are fetched only when an organiser opens those pages (Figure 3.1 in Chapter III).

2.3.2 Shared state: Context, custom hooks and useReducer (M1.4, M1.5, M1.11)

`CurrentUserProvider` holds the selected demo user and the user's profile. The context value is memoised, so consumers re-render only when the user or the profile actually changes, and the `useCurrentUser` hook hides `useContext` and guards against use outside the provider (Listing 2.4).

Listing 2.4: Memoised context value and the custom hook that reads it (`frontend/src/features/users/CurrentUserContext.jsx`, lines 59–79)

```
59    const value = useMemo(
60      () => ({
61        userId,
62        user: profile.data?.data ?? null,
63        isLoading: Boolean(userId) && profile.isPending,
64        isOrganiser: profile.data?.data?.role === 'ORGANISER',
65        setUserId,
66        signOut: () => setUserId(null),
67      }),
68      [userId, profile.data, profile.isPending, setUserId],
69    );
70
71    return <CurrentUserContext.Provider
          value={value}>{children}</CurrentUserContext.Provider>;
72  }
73
74  // [FULLSTACK M1.5: Custom hook] Components call useCurrentUser() instead of
       useContext(...) directly.
75  export function useCurrentUser() {
76    const ctx = useContext(CurrentUserContext);
77    if (!ctx) throw new Error('useCurrentUser must be used inside
          <CurrentUserProvider>');
78    return ctx;
79  }
```

The catalogue has five filters that change together: a category chip toggles, and "Clear filters" resets all of them. One reducer with named actions keeps these transitions in a pure function that is tested without rendering anything (FS-T01, FS-T02). `toQueryParams` drops empty values so that the query key stays small and stable (Listing 2.5).

Listing 2.5: Catalogue filters as a reducer (`frontend/src/features/events/state/filtersReducer.js`, lines 4–28)

```
 4  export const initialFilters = Object.freeze({ q: '', clubId: '', category: '',
        availability: '', sort: 'soonest' });
 5
 6  export function filtersReducer(state, action) {
 7    switch (action.type) {
 8      case 'search':
 9        return { ...state, q: action.value };
10      case 'club':
11        return { ...state, clubId: action.value };
12      case 'toggleCategory':
13        return { ...state, category: state.category === action.value ? '' :
              action.value };
        // … 4 lines omitted …
18      case 'reset':
19        return initialFilters;
20      default:
21        throw new Error(`Unknown filter action: ${action.type}`);
22    }
23  }
24
25  /** Only non-empty filters become query params (so the cache key stays small and
        stable). */
26  export function toQueryParams(filters) {
27    return Object.fromEntries(Object.entries(filters).filter(([, v]) => v !== ''
        && v !== undefined && v !== null));
28  }
```

A small custom hook debounces the search box: the value is released only after the user has stopped typing for 300 ms, so typing “hackathon” sends one request instead of nine (FS-T18).

Listing 2.6: Debounce as a reusable custom hook (`frontend/src/shared/hooks/useDebouncedValue.js`, lines 5–12)

```
 5  export function useDebouncedValue(value, delayMs = 300) {
 6    const [debounced, setDebounced] = useState(value);
 7    useEffect(() => {
 8      const timer = setTimeout(() => setDebounced(value), delayMs);
 9      return () => clearTimeout(timer);
10    }, [value, delayMs]);
11    return debounced;
12  }
```

2.3.3 Forms with React Hook Form and server errors (M1.6, M2.9)

Every event stores its own registration questions as JSON (for example a T-shirt size or whether the student brings a laptop). `QuestionField` turns each question into a registered input whose name is `answers.<questionId>`, with the validation rules the question asks for (Listing 2.7).

Listing 2.7: Form fields generated from the event's JSONB questions (`frontend/src/features/events/RegistrationPanel.jsx`, lines 17–54)

```
17  function QuestionField({ question, register, error }) {
18    const name = `answers.${question.id}`;
19    const rules = { required: question.required ? `"${question.label}" is
          required` : false };
      // … 13 lines omitted …
33    if (question.type === 'select') {
34      return (
35        <Field label={question.label} error={error}>
36          <select className="input" defaultValue="" {...register(name, rules)}>
37            <option value="" disabled>
38              Choose…
39            </option>
40            {question.options.map((o) => (
41              <option key={o} value={o}>
42                {o}
43              </option>
44            ))}
45          </select>
46        </Field>
47      );
48    }
49    return (
50      <Field label={question.label} error={error}>
51        <input className="input" maxLength={200} {...register(name, { ...rules,
              maxLength: { value: 200, message: 'Keep it under 200 characters' } })}
              />
52      </Field>
53    );
54  }
```

On submit, the form runs the register mutation. If the server rejects an answer, it returns 422 with errors such as `{ field: "answers.tshirt", message: "Choose one of: S, M, L" }`; `setError` places each message under the matching input, exactly as a client-side error would appear (Listing 2.8).

Listing 2.8: Submitting through a mutation and mapping server errors onto fields (`frontend/src/features/events/RegistrationPanel.jsx`, lines 74–90)

```
74  async function onSubmit(values) {
75    try {
76      const res = await registerMutation.mutateAsync(values.answers ?? {});
77      const { outcome, position } = res.data;
78      toast.show(outcome === 'CONFIRMED' ? "You're in! Seat confirmed." : `Event
            is full: you are #${position} on the waitlist.`, {
79        tone: outcome === 'CONFIRMED' ? 'success' : 'warning',
80      });
81      reset();
82    } catch (err) {
83      const fields = Object.entries(err.fieldErrors ?? {});
84      if (fields.length) {
85        for (const [field, message] of fields) setError(toFormField(field), {
              type: 'server', message });
86      } else {
87        toast.show(err.message, { tone: 'error' });
88      }
89    }
90  }
```

The organiser's event editor uses `useFieldArray` for its list of custom questions and custom `validate` rules for checks that involve more than one field: the capacity must fit the selected venue, and the event must end after it starts (Listing 2.9). Both errors appear without contacting the server (FS-T19).

Listing 2.9: Cross-field validation rules in the event editor (`frontend/src/features/organiser/EventFormPage.jsx`, lines 179–212)

```
<Field label="Capacity" error={errors.capacity?.message}>
  <input
    className="input"
    type="number"
    min={1}
    {...register('capacity', {
      required: 'Capacity is required',
      valueAsNumber: true,
      min: { value: 1, message: 'At least 1 seat' },
      max: { value: 2000, message: 'At most 2000 seats' },
      validate: (value) => !selectedVenue || value <= selectedVenue.capacity ||
          `The venue holds at most ${selectedVenue.capacity}`,
    })}
  />
</Field>
// … 10 lines omitted …
<Field label="Ends" error={errors.endsAt?.message}>
  <input
    className="input"
    type="datetime-local"
    {...register('endsAt', {
      required: 'End time is required',
      validate: (v) => new Date(v) > new Date(getValues('startsAt')) || 'The
          event must end after it starts',
    })}
  />
</Field>
```

2.3.4 One Axios instance with interceptors (M2.1, M2.2, M2.10)

All API calls go through `apiClient` (Listing 2.10). Its base URL comes from `VITE_API_URL`. The request interceptor attaches the current user's id; the response interceptor converts every failure into an `ApiError` with a status, a stable code, per-field errors and the server's request id, and sends the user to the "choose user" page on a 401.

Listing 2.10: Axios instance, request and response interceptors (`frontend/src/services/apiClient.js`, lines 18–62)

```
18  export const apiClient = axios.create({
19    baseURL: import.meta.env.VITE_API_URL ?? 'http://localhost:4000/api/v1',
20    timeout: 10_000,
21    headers: { Accept: 'application/json' },
22  });
23
    // … 7 lines omitted …
31
32  apiClient.interceptors.request.use((config) => {
33    const userId = getUserId();
34    if (userId) config.headers['X-User-Id'] = String(userId);
35    return config;
36  });
37
38  export function toApiError(error) {
39    if (error instanceof ApiError) return error;
40    if (!error.response) {
41      return new ApiError({ status: 0, code: 'NETWORK_ERROR', message: 'Cannot
          reach the SeatSure API. Is the server running?' });
42    }
43    const { status, data, headers } = error.response;
44    const fieldErrors = Object.fromEntries((data?.errors ?? []).map((e) =>
        [e.field, e.message]));
45    return new ApiError({
46      status,
47      code: data?.code ?? 'HTTP_ERROR',
48      message: data?.message ?? `Request failed (${status})`,
49      fieldErrors,
50      requestId: data?.requestId ?? headers?.['x-request-id'],
51    });
52  }
53
54  apiClient.interceptors.response.use(
55    (response) => response,
56    (error) => {
57      const apiError = toApiError(error);
58      // 401 = no/unknown demo user: send the person to the "choose user" screen.
59      if (apiError.status === 401) onUnauthorized(apiError);
60      return Promise.reject(apiError);
61    },
62  );
```

The API only accepts browser requests from the React origins listed in its CORS configuration (M2.1), and it exposes the custom headers the client reads, such as `X-Request-Id`:

Listing 2.11: CORS allow-list on the Express side (`backend/src/app.js`, lines 38–44)

```
38  app.use(
39    // 3. [FULLSTACK M2.1: CORS] only the React dev/prod origins may call the API
        from a browser
40    cors({
41      origin: config.corsOrigins,
42      exposedHeaders: ['X-Request-Id', 'X-Response-Time', 'Deprecation', 'Sunset',
          'Link', 'Location'],
43    }),
44  );
```

Query retries are limited to failures that a retry can fix: a network error or a 5xx response. A 404 or 409 will not change by asking again, and mutations are never retried automatically (Listing 2.12).

Listing 2.12: QueryClient defaults and the retry policy (`frontend/src/app/queryClient.js`, lines 5–15)

```
5  return new QueryClient({
6    defaultOptions: {
7      queries: {
8        staleTime: 10_000,
9        refetchOnWindowFocus: true,
10       // Retry only failures that a retry can fix: the network or a 5xx/503.
             Never retry 4xx
11       // (a 404 or 409 will not change by asking again).
12       retry: (failureCount, error) => failureCount < 2 && (error?.status === 0
             || error?.status >= 500),
13       retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 4000),
14     },
15     mutations: { retry: false },
```

2.3.5 Caching, cursor pagination and infinite scroll (M2.3–M2.5, M2.11, M3.9, M3.11, M3.12)

Every cache key comes from one factory (Listing 2.13). Keys that depend on who is asking include the user id, so switching the demo user never shows another person's cached data, and invalidating `['events', 'list']` refreshes every filtered catalogue at once.

Listing 2.13: Query-key factory (`frontend/src/shared/lib/queryKeys.js`, lines 4–20)

```
4  export const queryKeys = {
5    events: {
6      all: ['events'],
7      lists: () => ['events', 'list'],
8      list: (filters) => ['events', 'list', filters],
9      detail: (id, userId) => ['events', 'detail', Number(id), userId ??
           'anonymous'],
10     activity: (id) => ['events', 'activity', Number(id)],
11     attendeesOf: (id) => ['events', 'attendees', Number(id)],
12     attendees: (id, params) => ['events', 'attendees', Number(id), params],
13     mine: (userId, params) => ['events', 'mine', userId, params],
14   },
15   me: {
16     registrations: (userId) => ['me', userId, 'registrations'],
17     tasks: (userId) => ['me', userId, 'tasks'],
18     notifications: (userId) => ['me', userId, 'notifications'],
19     profile: (userId) => ['me', userId, 'profile'],
20   },
```

`useEventsInfinite` wraps `useInfiniteQuery`. Each page is requested with the previous page's `nextCursor`; the data stays fresh for 15 s and the list is re-polled every 30 s so that seat counts stay current (Listing 2.14).

Listing 2.14: Cursor pagination with useInfiniteQuery
(`frontend/src/features/events/hooks/useEvents.js`, lines 14–25)

```
14  export function useEventsInfinite(params) {
15    return useInfiniteQuery({
16      queryKey: queryKeys.events.list(params),
17      queryFn: ({ pageParam }) => eventsApi.list(params, pageParam),
18      initialPageParam: undefined,
19      getNextPageParam: (lastPage) => lastPage.meta?.nextCursor ?? undefined,
20      // [FULLSTACK M2.4 + M3.12: Freshness] Catalogue data may be 15 s old; seat
            counts refresh every 30 s.
21      staleTime: 15_000,
22      refetchInterval: 30_000,
23      placeholderData: keepPreviousData, // keep showing old results while new
            filters load
24    });
25  }
```

On the page, the reducer, the debounce and `useMemo` produce a stable parameter object; `useCallback` keeps `loadMore` stable; and the intersection-observer hook calls it when the sentinel becomes visible (Listing 2.15, Listing 2.16).

Listing 2.15: Catalogue page: filters, stable parameters and the infinite-scroll trigger
(`frontend/src/features/events/EventsPage.jsx`, lines 11–26)

```
11  const [filters, dispatch] = useReducer(filtersReducer, initialFilters);
12  const debouncedQ = useDebouncedValue(filters.q, 300);
13  const { clubId, category, availability, sort } = filters;
14
15  // [FULLSTACK M1.7: useMemo] A stable params object -> a stable query key -> no
        accidental refetches.
16  const params = useMemo(() => toQueryParams({ clubId, category, availability,
        sort, q: debouncedQ.trim() }), [clubId, category, availability, sort,
        debouncedQ]);
17
18  const { data: clubs } = useClubs();
19  const { data, error, isPending, isFetching, hasNextPage, fetchNextPage,
        isFetchingNextPage, refetch } = useEventsInfinite(params);
20  const events = useMemo(() => data?.pages.flatMap((page) => page.data) ?? [],
        [data]);
21
22  // [FULLSTACK M3.11: Infinite scroll] When the sentinel below the grid becomes
        visible, load the next page.
23  const loadMore = useCallback(() => {
24    if (hasNextPage && !isFetchingNextPage) fetchNextPage();
25  }, [hasNextPage, isFetchingNextPage, fetchNextPage]);
26  const sentinelRef = useIntersectionObserver(loadMore, { enabled:
        Boolean(hasNextPage) });
```

Listing 2.16: IntersectionObserver wrapped in a custom hook (`frontend/src/shared/hooks/useIntersectionObserver.js`, lines 5–24)

```
 5  export function useIntersectionObserver(onIntersect, { enabled = true,
        rootMargin = '300px' } = {}) {
 6    const targetRef = useRef(null);
 7    const callbackRef = useRef(onIntersect);
 8    callbackRef.current = onIntersect;
 9
10    useEffect(() => {
11      const node = targetRef.current;
12      if (!enabled || !node) return undefined;
13      const observer = new IntersectionObserver(
14        (entries) => {
15          if (entries.some((entry) => entry.isIntersecting))
                callbackRef.current();
16        },
17        { rootMargin },
18      );
19      observer.observe(node);
20      return () => observer.disconnect();
21    }, [enabled, rootMargin]);
22
23    return targetRef;
24  }
```

On the server, the cursor is resolved by Prisma. The repository asks for one row more than the page size: if that extra row exists there is another page, and the id of the last row on this page becomes `nextCursor` (Listing 2.17).

Listing 2.17: Server half of cursor pagination (Prisma) (`db/src/repositories/eventRepository.ts`, lines 63–73)

```
63  const rows = await client.event.findMany({
64    where,
65    select: cardSelect,
66    orderBy: ORDER_BY[query.sort ?? 'soonest'],
67    take: limit + 1,
68    ...(query.cursor ? { cursor: { id: query.cursor }, skip: 1 } : {}),
69  });
70
71  const hasMore = rows.length > limit;
72  const items = hasMore ? rows.slice(0, limit) : rows;
73  return { items, nextCursor: hasMore ? items[items.length - 1]!.id : null };
```

2.3.6 Mutations, selective invalidation and optimistic updates (M2.6–M2.8)

After a registration succeeds, `useRegister` refreshes exactly the four caches whose data changed: this event (seats and my status), the catalogue lists (seat counts), “My passes” and the event’s activity feed (Listing 2.18). Registration is deliberately *not* optimistic, because only the server can decide between a confirmed seat and a waitlist place.

Listing 2.18: Register mutation with selective invalidation
(`frontend/src/features/events/hooks/useEvents.js`, lines 61–73)

```
61  export function useRegister(eventId) {
62    const queryClient = useQueryClient();
63    const { userId } = useCurrentUser();
64    return useMutation({
65      mutationFn: (answers) => registrationsApi.register(eventId, answers),
66      onSuccess: () => {
67        queryClient.invalidateQueries({ queryKey: queryKeys.events.detail(eventId,
              userId) });
68        queryClient.invalidateQueries({ queryKey: queryKeys.events.lists() });
69        queryClient.invalidateQueries({ queryKey:
              queryKeys.me.registrations(userId) });
70        queryClient.invalidateQueries({ queryKey:
              queryKeys.events.activity(eventId) });
71      },
72    });
73  }
```

Cancellation, whose outcome the user already knows, is optimistic (Listing 2.19). `onMutate` pauses refetches, takes a snapshot and writes the cancelled state into the cache; `onError` restores the snapshot; `onSettled` refetches so that the cache always ends up matching the server.

Listing 2.19: Optimistic cancellation with rollback
(`frontend/src/features/registrations/hooks/useMyRegistrations.js`, lines 24–56)

```
24  export function useCancelRegistration() {
25    const queryClient = useQueryClient();
26    const { userId } = useCurrentUser();
27    const key = queryKeys.me.registrations(userId);
28
29    return useMutation({
30      mutationFn: ({ registrationId }) => registrationsApi.cancel(registrationId),
31      onMutate: async ({ registrationId }) => {
32        await queryClient.cancelQueries({ queryKey: key });
33        const previous = queryClient.getQueryData(key);
34        queryClient.setQueryData(key, (old) =>
35          old
36            ? {
37                ...old,
38                data: old.data.map((r) => (r.id === registrationId ? { ...r,
                      status: 'CANCELLED', position: null, cancelledAt: new
                      Date().toISOString() } : r)),
39              }
40            : old,
41        );
42        return { previous };
43      },
44      onError: (_err, _vars, context) => {
45        if (context?.previous) queryClient.setQueryData(key, context.previous);
46      },
47      onSettled: (_data, _err, { eventId }) => {
48        queryClient.invalidateQueries({ queryKey: key });
          // … 4 lines omitted …
53        queryClient.invalidateQueries({ queryKey: queryKeys.events.lists() });
54      },
55    });
56  }
```

2.3.7 Measured memoisation (M1.7)

The catalogue polls every 30 s. TanStack Query's structural sharing returns the *same objects* for events that did not change, so wrapping `EventCard` in `React.memo` (`export const EventCard = memo(EventCardComponent)`, line 30 of `EventCard.jsx`) should stop those cards from re-rendering. Following the course rule of measuring first, test FS-T27 counts renders: `SeatMeter` is rendered once per card, so its call count is the number of card renders (Listing 2.20).

Listing 2.20: Render-count test proving that React.memo works (`frontend/src/features/events/EventCard.test.jsx`, lines 28–46)

```
28  function List() {
29    const [events, set] = useState([event(1), event(2), event(3)]);
30    setEvents = set;
31    return events.map((e) => <EventCard key={e.id} event={e} />);
32  }
33  render(
34    <MemoryRouter>
35      <List />
36    </MemoryRouter>,
37  );
38  expect(SeatMeter).toHaveBeenCalledTimes(3); // first render: 3 cards
39
40  // A poll returns the same objects (structural sharing): no card renders again.
41  act(() => setEvents((list) => [...list]));
42  expect(SeatMeter).toHaveBeenCalledTimes(3);
43
44  // Someone registers for event 2: only that card renders again.
45  act(() => setEvents((list) => list.map((e) => (e.id === 2 ? event(2, 3) : e))));
46  expect(SeatMeter).toHaveBeenCalledTimes(4);
```

2.3.8 Error boundaries around risky panels (M1.9)

The analytics page draws several independent reports. Each one is wrapped in a `Panel`: an error boundary combined with TanStack Query's `QueryErrorResetBoundary`, so that "Try again" both re-renders the panel and retries its failed query (Listing 2.21). A "Simulate a chart crash" button makes the sign-ups chart throw during rendering (`if (crash) throw new Error(…)`, line 27), which demonstrates that only that panel shows a fallback (FS-T22, Figure 3.9).

Listing 2.21: A reusable boundary that also resets failed queries (`frontend/src/features/organiser/AnalyticsPage.jsx`, lines 14–24)

```
14  function Panel({ children, resetKeys }) {
15    return (
16      <QueryErrorResetBoundary>
17        {({ reset }) => (
18          <ErrorBoundary FallbackComponent={ErrorFallback} onReset={reset}
                resetKeys={resetKeys}>
19            {children}
20          </ErrorBoundary>
21        )}
22      </QueryErrorResetBoundary>
23    );
24  }
```

2.3.9 Composition and nested data (M1.12, M3.3–M3.5)

`Card` is a compound component: callers compose `Card.Header`, `Card.Body` and `Card.Footer`, and the header offers named slots (`eyebrow`, `title`, `actions`). The same component renders event cards, passes, forms and analytics panels (Listing 2.22).

Listing 2.22: Compound component with named slots (`frontend/src/shared/components/Card.jsx`, lines 4–35)

```
 4  export function Card({ children, as: Tag = 'article', className = '', ...rest })
        {
 5    return (
 6      <Tag className={`card ${className}`} {...rest}>
 7        {children}
 8      </Tag>
 9    );
10  }
11
12  function Header({ title, eyebrow, actions, children }) {
13    return (
        // … 8 lines omitted …
22    );
23  }
24
    // … 7 lines omitted …
32
33  Card.Header = Header;
34  Card.Body = Body;
35  Card.Footer = Footer;
```

A pass on the “My passes” page carries data from one nested Prisma `include`: registration → event → club and venue. `PassCard` renders it defensively with optional chaining and fallbacks, so a missing relation never crashes the page (FS-T13, Listing 2.23).

Listing 2.23: Rendering nested include data safely (`frontend/src/features/registrations/PassCard.jsx`, lines 10–23)

```
10  const event = pass.event ?? {};
11  const upcoming = new Date(event.startsAt) > new Date();
12  const canCancel = upcoming && pass.status !== 'CANCELLED';
13
14  return (
15    <Card className={`pass pass--${pass.status.toLowerCase()}`}>
16      <Card.Header
17        eyebrow={event.club?.name ?? 'Club'}
18        title={<Link to={`/events/${event.id}`}>{event.title ?? 'Event'}</Link>}
19        actions={<Badge>{pass.status}</Badge>}
20      />
21      <Card.Body>
22        <p>{formatRange(event.startsAt, event.endsAt)}</p>
23        <p className="muted">{event.venue ? `${event.venue.name},
              ${event.venue.building}` : 'Venue to be announced'}</p>
```

Database errors reach the user as readable messages. When a student registers twice, PostgreSQL’s partial unique index rejects the insert, Prisma reports P2002, and the API’s error handler maps the constraint name to 409 `ALREADY_REGISTERED` with the message “You are already registered for this event”, which the client shows as a toast (Listing 2.24).

Listing 2.24: Constraint names mapped to user-facing messages (server side) (`backend/src/middleware/errorHandler.js`, lines 13–26)

```
13  const CONSTRAINT_ERRORS = {
14    registrations_one_active_per_user_key: ['You are already registered for this
          event', 'ALREADY_REGISTERED'],
15    events_confirmed_within_capacity_chk: ['This event is already full',
          'EVENT_FULL'],
16    events_time_order_chk: ['The event must end after it starts',
          'INVALID_TIME_RANGE'],
17    users_email_key: ['That email address is already in use', 'EMAIL_TAKEN'],
18    clubs_name_key: ['A club with that name already exists', 'DUPLICATE_CLUB'],
19    tasks_title_not_blank_chk: ['A task needs a title', 'TITLE_REQUIRED'],
20  };
21
22  function fromDbError(dbErr) {
23    const [message, code] = CONSTRAINT_ERRORS[dbErr.constraint] ?? [];
24    switch (dbErr.kind) {
25      case 'UNIQUE_VIOLATION': // Prisma P2002
26        return new AppError(message ?? 'This record already exists', 409, code ??
              'DUPLICATE');
```

Organiser tables use offset pagination instead: the attendee list sends `page` and `pageSize`, the API answers with `meta.page`, `meta.total` and `meta.totalPages`, and a shared `Pagination` component renders “Page 2 of 5” with Previous and Next buttons (M3.8). Polling settings for the remaining data (M3.12) are: the event page every 5 s, the notification bell every 15 s, and “My passes” on every window focus with `staleTime: 0`, because a pass can change when *someone else* cancels and the student is promoted.

2.3.10 CRUD over REST in the interface: My tasks (M1.6, M2.6–M2.9)

The “My tasks” page is the simplest end-to-end example of CRUD over REST in the client. All four operations go through one small service module (Listing 2.25): the list is a query, and adding, editing, ticking done and deleting are mutations. The add form checks the title with React Hook Form before anything is sent (FS-T29); ticking a task done is optimistic, using the same snapshot-and-rollback pattern as cancelling a pass (FS-T30); and a 400 from the server appears under the field it concerns (FS-T32).

Listing 2.25: The four REST calls behind “My tasks” (`frontend/src/services/tasksApi.js`, lines 4–9)

```
4   export const tasksApi = {
5     list: () => apiClient.get('/tasks').then(unwrap),
6     create: (task) => apiClient.post('/tasks', task).then(unwrap),
7     update: (id, changes) => apiClient.patch(`/tasks/${id}`,
          changes).then(unwrap),
8     remove: (id) => apiClient.delete(`/tasks/${id}`).then(() => null),
9   };
```

2.4 Connection with Back-End Web Development and Advanced Database Systems

The client depends on the other two parts of SeatSure only through explicit contracts. Figure 2.2 follows one registration through all three layers.

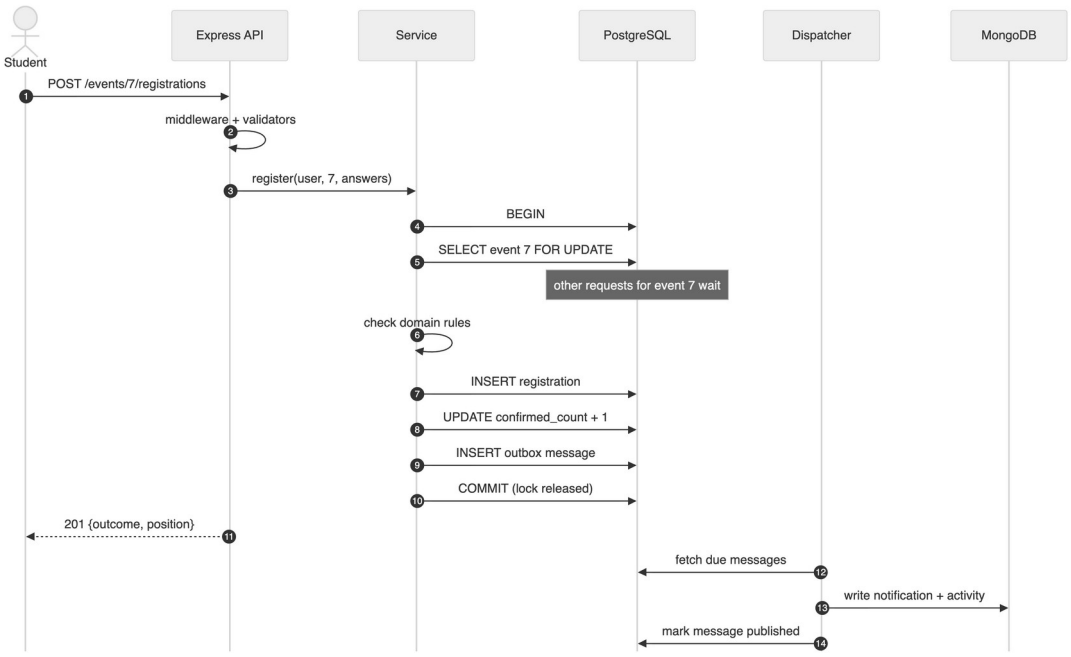


Figure 2.2: One registration across the client, the API and the two databases

- **The API envelope (Back-End).** Every success response has the shape `{status, data, meta}` and every error `{status, code, message, errors?, requestId}`. Because this contract never changes, `toApiError` can turn any failure into an `ApiError`, and React Hook Form can map `errors[].field` paths such as `answers.tshirt` straight onto inputs.
- **Business decisions stay on the server (Back-End).** The client never decides whether a seat is free. It shows the server's answer, `CONFIRMED` or `WAITLISTED` with a position, which was decided inside a PostgreSQL transaction that holds a row lock on the event. This is why registration is not optimistic.
- **Pagination metadata (Back-End and Database).** Infinite scroll follows `meta.nextCursor`, which the server computes by asking PostgreSQL for `limit + 1` rows ordered by `(starts_at, id)`, the same order as the partial index `events_published_starts_at_idx`, so every page is served by an index scan.
- **Constraints become messages (Database).** Unique-index, foreign-key and CHECK violations are translated by the API into stable codes such as `ALREADY_REGISTERED` and `EVENT_FULL`, which the client displays.

- **JSONB drives the interface (Database).** The registration form is generated from the event's `questions` column (JSONB), and the organiser's attendee filter sends `answers[tshirt]=M`, which PostgreSQL answers with a GIN-indexed containment (`@>`) query (Figure 3.8).
- **MongoDB features degrade gracefully (Back-End and Database).** Notifications and the activity feed come from MongoDB. When MongoDB is unavailable the API returns `meta.degraded: true` and the client shows a notice instead of an error page, while registration keeps working on PostgreSQL.

2.5 Setting Up and Running the Project

The project needs Node.js 22 and PostgreSQL 18. MongoDB is optional in development: when no MongoDB address is configured, an npm-distributed MongoDB server starts automatically. From the `seatsure` folder:

1. Install the dependencies of all three workspaces (`frontend`, `backend`, `db`): `npm install`.
2. Copy `.env.example` to `.env` and `frontend/.env.example` to `frontend/.env`, then set `DATABASE_URL` and `DATABASE_URL_TEST` to your PostgreSQL user.
3. Create, migrate and seed the databases: `npm run setup` (it runs `db:create`, `db:migrate` and `db:seed`). Alternatively, load `db/schema.sql` with `psql` and run `npm run db:seed`.
4. Start the backend and the frontend together: `npm run dev`. The API listens on http://localhost:4000 and the React app on http://localhost:5173.
5. Open http://localhost:5173 in a browser and choose a demo user in the header, for example Priya Singh (student) or Aarav Mehta (organiser).
6. Run the automated tests with `npm test`, and the live API checks with `npm run test:api` while the API is running.

If PostgreSQL cannot be installed, the README explains how to use free hosted databases instead (Neon or Supabase for PostgreSQL, MongoDB Atlas for MongoDB).

Table 2.2 is a short reference of the main routes. Every route is under `/api/v1` and answers with the same JSON envelope: `{status, data, meta}` on success and `{status, code, message, errors?, requestId}` on failure. Requests that need a user send the `X-User-Id` header.

Table 2.2: Short API reference (all routes under `/api/v1`)

Method	Route	Purpose	Success
GET	`/tasks`	My tasks, open ones first, with counts	200
POST	`/tasks`	Create a task `{title, notes?, dueDate?}`	201 + Location
PATCH	`/tasks/:id`	Edit a task or tick it done `{title?, notes?, dueDate?, completed?}`	200
DELETE	`/tasks/:id`	Delete a task (another user's or a missing task: 404)	204
GET	`/events`	Upcoming events with filters and cursor pagination	200
GET	`/events/:id`	One event, its free seats and my registration	200
POST	`/events`	Create an event draft (organiser)	201 + Location
PATCH	`/events/:id`	Edit, publish or cancel an event (organiser)	200
DELETE	`/events/:id`	Delete a draft (organiser)	204
POST	`/events/:id/registrations`	Register: CONFIRMED or WAITLISTED	201 + Location
GET	`/me/registrations`	My passes with waitlist positions	200
DELETE	`/registrations/:id`	Cancel; promotes the next waitlisted student	200
PATCH	`/registrations/:id`	Check an attendee in (organiser)	200
GET	`/events/:id/registrations`	Attendee list with filters (organiser)	200
GET	`/analytics/overview`	Organiser analytics (window functions)	200
GET	`/health`	Status of PostgreSQL, MongoDB and the outbox	200

2.6 CRUD over REST: Route Mapping and Validation

SeatSure exposes its data as REST resources, so each create, read, update and delete operation is one HTTP method on one URL. Table 2.3 shows the mapping for its three resources (tasks, events and registrations), together with the database operation behind each route and the status codes a client can expect. Tasks are the plainest example: a personal to-do list with no business rules beyond validation and ownership.

Table 2.3: CRUD operations mapped to REST routes (under `/api/v1`)

Operation	Route	Database operation	Success	Typical errors

Create	`POST /tasks`
	`POST /events`
	`POST /events/:id/registrations`
		`task.create`
		`event.create` (draft)
		INSERT inside a transaction that locks the event row
			201 + `Location`	400, 403, 409, 422

Read	`GET /tasks`, `GET /tasks/:id`
	`GET /events`, `GET /events/:id`
	`GET /me/registrations`
		`findMany` scoped to the user
		`findMany` with a cursor
		`findUnique` / `include` for related data
			200	400, 404

Operation	Route	Database operation	Success	Typical errors

Update	`PATCH /tasks/:id`
	`PATCH /events/:id`
	`PATCH /registrations/:id`
		`task.update` (edit, tick done)
		`event.update` (edit, publish, cancel)
		`updateMany` for an idempotent check-in
			200	400, 403, 404, 409

Delete	`DELETE /tasks/:id`
	`DELETE /events/:id`
	`DELETE /registrations/:id`
		`task.delete`
		`event.delete` (drafts only)
		Status set to CANCELLED (soft delete) plus promotion
			204
			204
			200
				404, 409

Deleting a registration is a *soft* delete. The row is kept with status CANCELLED, so the history stays available for analytics, and the partial unique index ignores cancelled rows, so the student may register again. The response is 200 with a body rather than 204, because the client needs to know who was promoted into the freed seat.

Why validation runs on both the client and the server. The same input is checked in three places, each for a different reason:

- **In the browser:** React Hook Form checks required fields, lengths and cross-field rules (the end must come after the start; the capacity must fit the venue) before any request is sent. The user gets instant feedback and no request is wasted.
- **On the server:** the browser is under the user's control, and anyone can bypass the form with `curl` or the browser's developer tools. The API therefore validates every request again with express-validator. It answers 400 with a list of the invalid fields (live checks API-02 and API-10), drops fields a client may not set (mass assignment), and applies the business rules (409 or 422).
- **In the database:** CHECK, UNIQUE and FOREIGN KEY constraints protect the rules that must never break, such as `confirmed_count <= capacity`, even if a bug ever slips past the code above.

In short, client-side validation makes the form pleasant to use, while server-side and database validation make the system safe.

Chapter III – Results

3.1 Testing Approach

The client was tested at three levels:

- **Automated component and hook tests (32).** Vitest runs them with React Testing Library in a jsdom browser environment. The `services/` modules are mocked, so each test checks one React behaviour in isolation: forms, cache updates, routing, error handling and rendering performance.
- **Integration through the real API.** The Back-End report's Supertest tests call the same endpoints that the client calls, against PostgreSQL and MongoDB.
- **End-to-end evidence.** Screenshots of the running application, captured with headless Chrome against the real API and databases, show each feature working (FS-E01 to FS-E13 in Appendix A).
- **Live API checks (14).** `npm run test:api` sends one real request for every CRUD operation and error case to the running API and records the actual result (Section 3.4).

Every test case is recorded with an ID, the concept and lesson it proves, the scenario, the input, and the expected and actual results (Appendix A). The raw output of the run is saved in `docs/test-results/fullstack-tests.txt`.

3.2 Test Results Summary

Table 3.1: Automated client tests by area (27 Oct 2026, `npm run test:frontend`)

Area (lessons)	Test IDs	Tests	Result
Filters, reducer and custom hooks (M1.5, M1.11, M2.5)	FS-T01 – T04	4	4 passed
Axios instance and error handling (M2.2, M2.10)	FS-T05 – T08	4	4 passed
Forms: React Hook Form, `useFieldArray`, server errors (M1.6, M2.9)	FS-T09, T10, T19 – T21	5	5 passed
Mutations, optimistic update, invalidation (M2.6 – M2.8)	FS-T11, T14, T15	3	3 passed
Context and nested data (M1.4, M3.5)	FS-T12, T13	2	2 passed
Cursor pagination, infinite scroll, filters (M3.9, M3.11, M2.5)	FS-T16 – T18	3	3 passed
Error boundaries and degraded data (M1.9)	FS-T22, T23	2	2 passed
Routing and code splitting (M1.8, M1.10)	FS-T24 – T26	3	3 passed
Rendering performance (M1.7)	FS-T27	1	1 passed
Tasks: list, create, optimistic tick, delete, server errors (M1.6, M2.6–M2.9)	FS-T28 – T32	5	5 passed

Area (lessons)	Test IDs	Tests	Result
Total		**32**	**32 passed**

3.3 Results

The main results are listed below; the full table is in Appendix A.

- **All 32 automated tests passed** in 2.7 s on 27 October 2026.
- **Tasks CRUD (FS-T28–T32):** the list showed To do / Done counts; an empty title was stopped before any request; a new task was sent as `{title, notes, dueDate}` and the list refetched; a tick was shown before the server answered and undone when it failed; and a server error appeared under the edited field.
- **Optimistic update with rollback (FS-T14):** the pass moved to "Cancelled" before the server responded; when the server answered 409, it returned to "Upcoming" and an error toast appeared.
- **Infinite scroll (FS-T16):** when the sentinel became visible, the second request carried `cursor = 2`, the list grew to three events and the end-of-list message appeared.
- **Debounced search (FS-T18):** typing "git" (three keystrokes) produced exactly one request, with `q = git`.
- **Server errors on fields (FS-T10, FS-T21):** a 422 response for `answers.tshirt`, and one for `capacity` on the event form, appeared under the matching inputs.
- **Measured memoisation (FS-T27):** with three cards, a poll that returned the same objects caused no card to re-render (3 → 3 renders), and a change to one event re-rendered only that card (4 renders).
- **Contained failures (FS-T22, FS-T23):** a simulated chart crash showed a fallback inside one panel while the rest of the dashboard stayed usable, and a degraded MongoDB response showed a warning instead of an error page.
- **Code splitting:** a student's first visit downloads 126.7 KB of JavaScript (gzipped); the organiser area adds 23.7 KB only when it is opened (Figure 3.1).

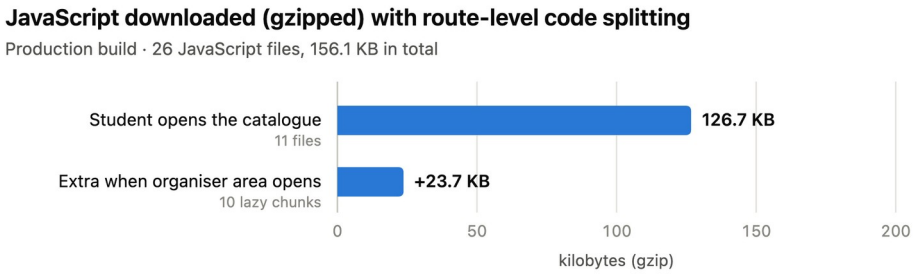


Figure 3.1: JavaScript downloaded with route-level code splitting (production build, gzipped)

The screenshots below were taken from the running application, connected to the real API and databases. Figure 3.2 shows the four task operations through the user interface, each a single REST request (Section 2.6), and Figure 3.3 the validation case.

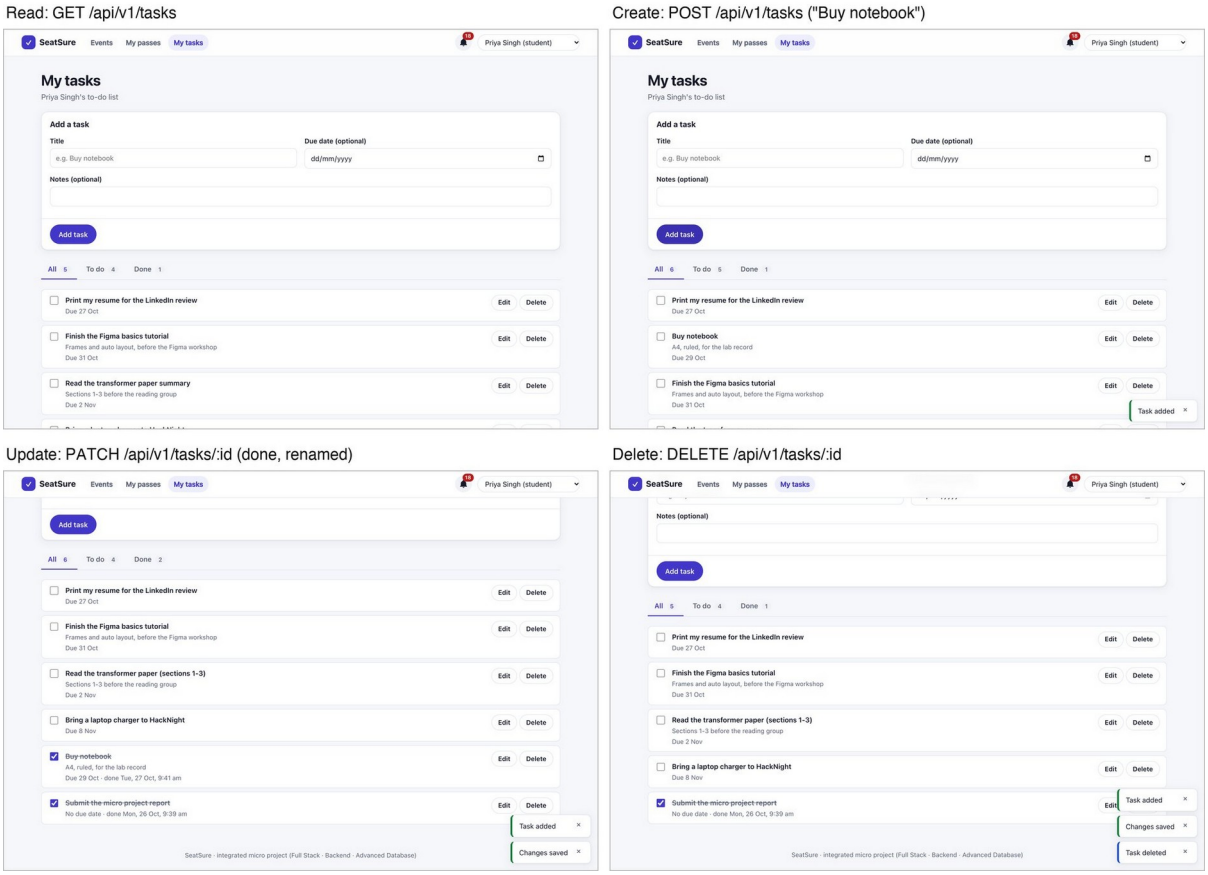


Figure 3.2: Tasks: read, create, update (tick done, rename) and delete through the user interface, each backed by one REST request

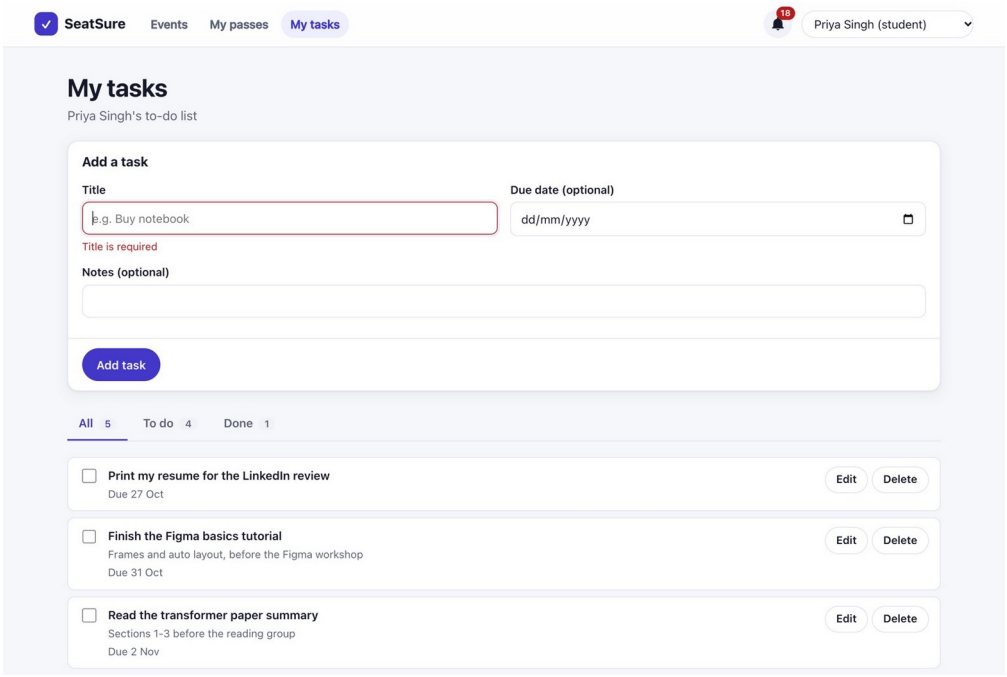


Figure 3.3: Validation in the browser: an empty title is stopped before any request is sent (the API would answer 400)

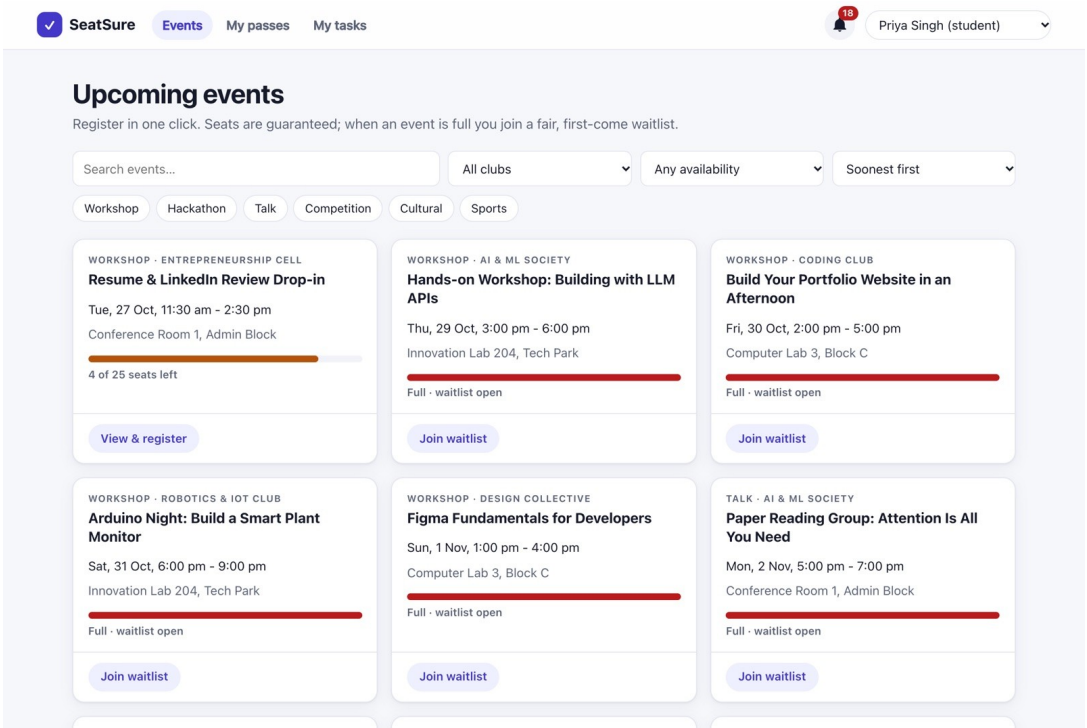


Figure 3.4: Catalogue with live seat meters, filters and infinite scroll (FS-E01)

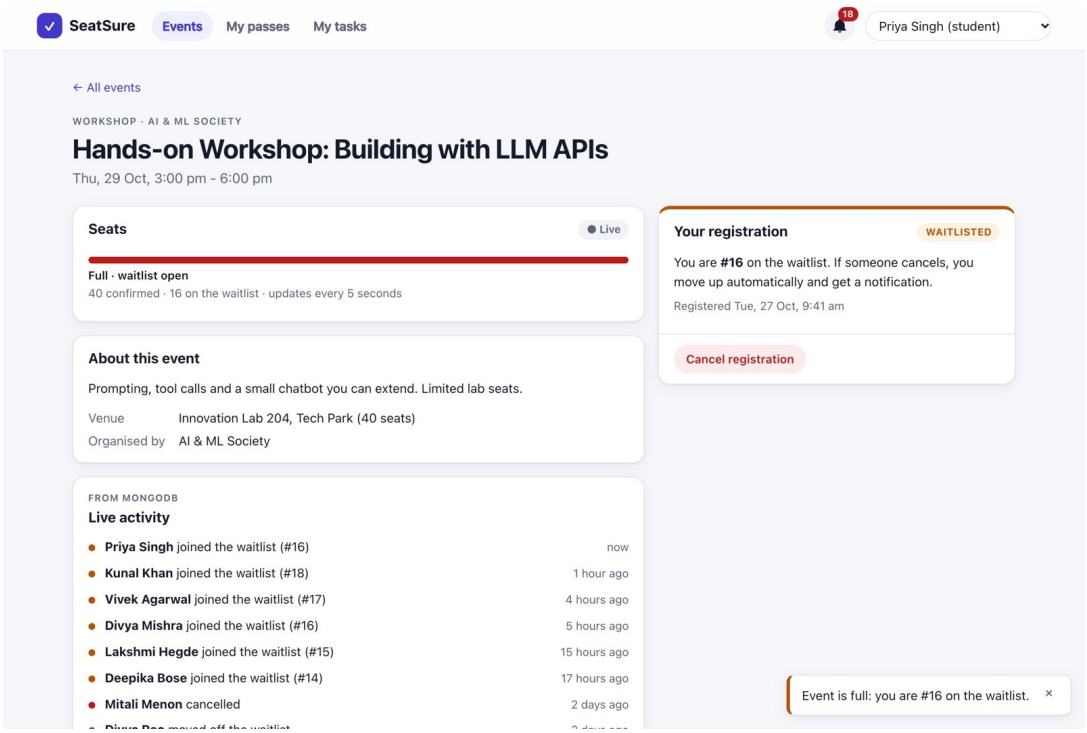


Figure 3.5: Registering for a full event: the server's waitlist position is shown in a toast, and the MongoDB activity feed updates (FS-E04)

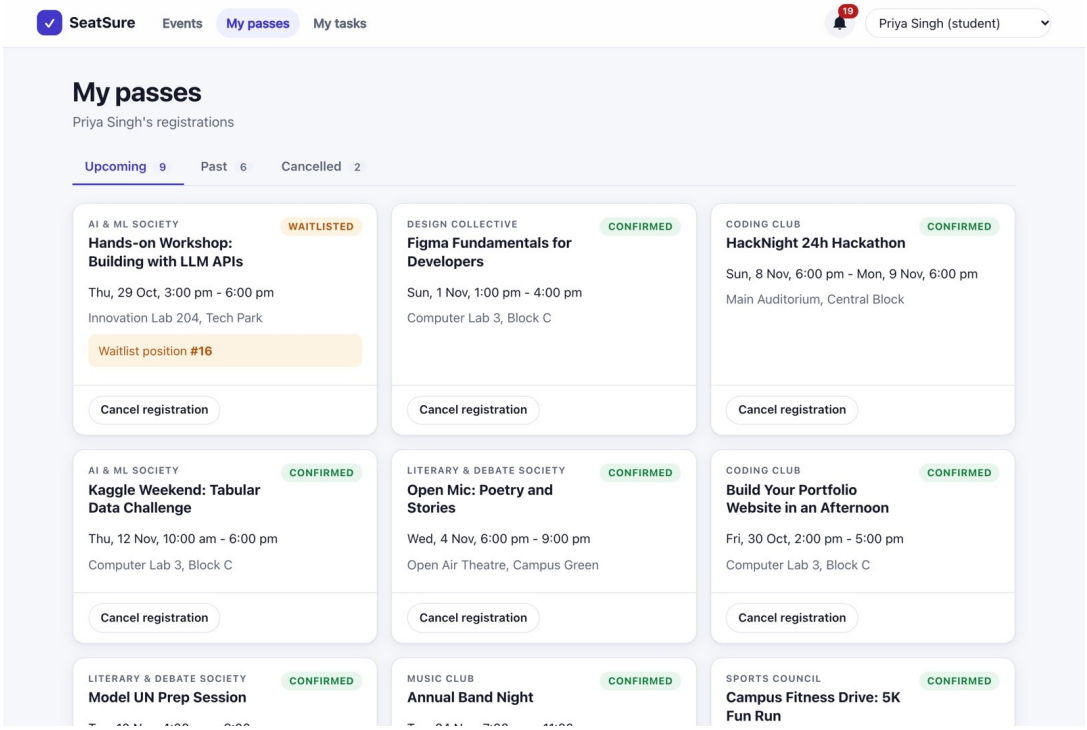


Figure 3.6: "My passes": nested include data (registration → event → club, venue) with waitlist positions (FS-E05)

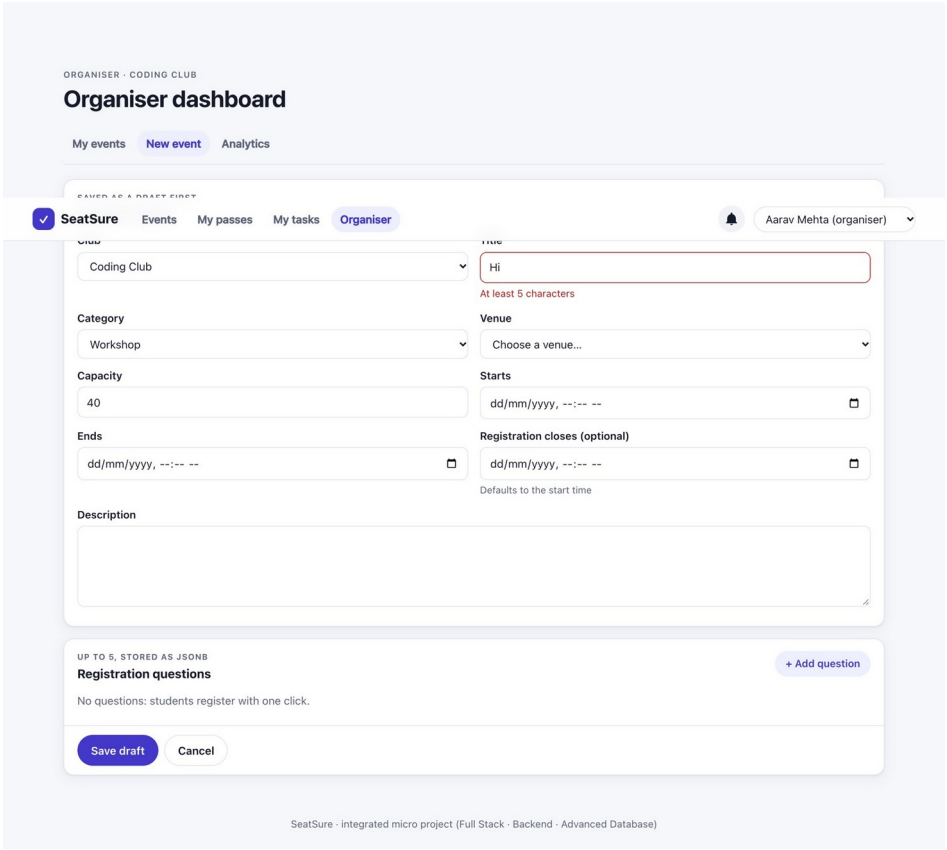


Figure 3.7: Event editor: React Hook Form validation, including cross-field rules (FS-E08)

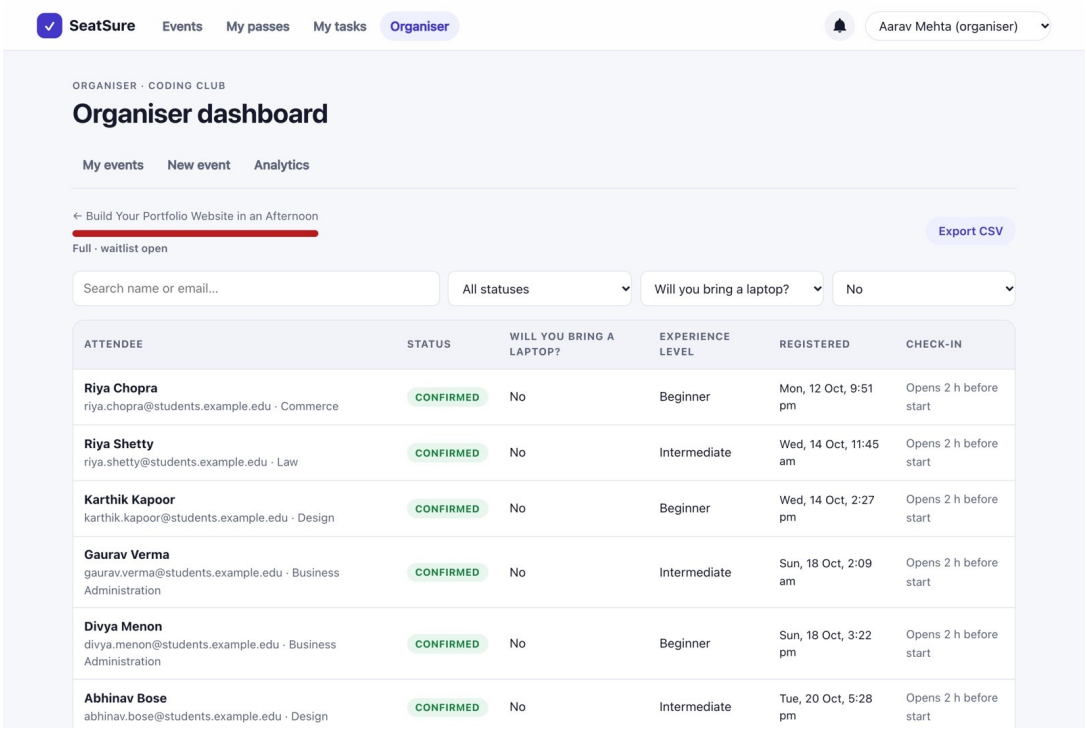


Figure 3.8: Organiser attendee list filtered by a JSONB answer, with offset pagination (FS-E09)

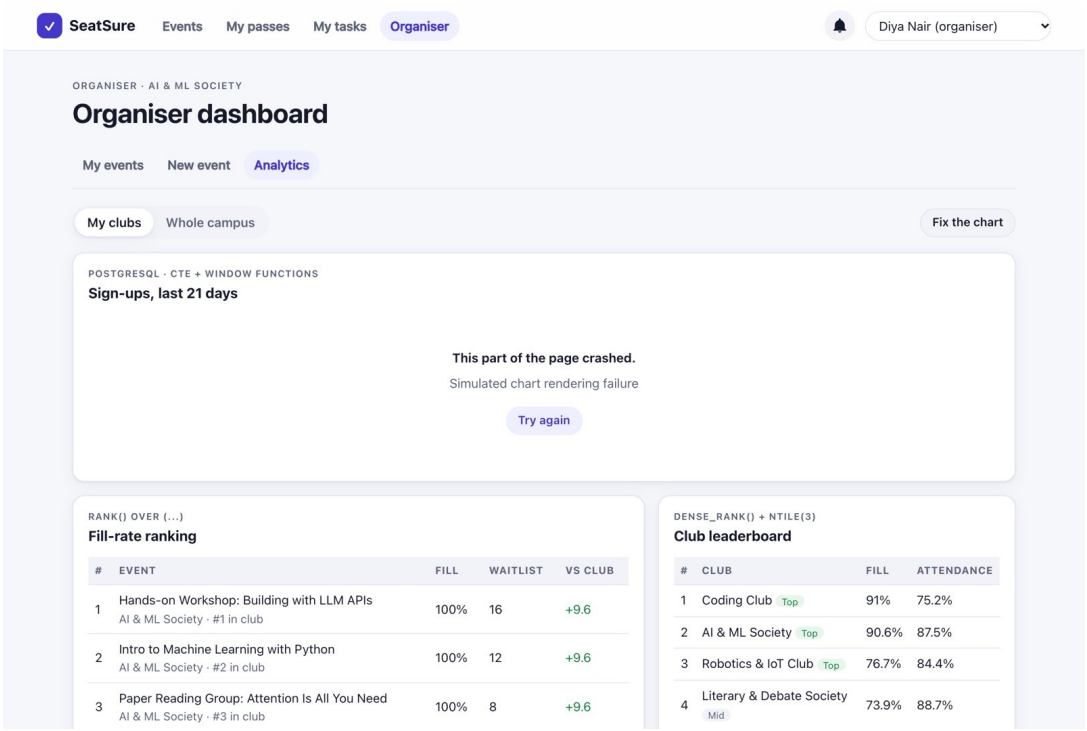


Figure 3.9: A crashing chart is contained by its own error boundary; the rest of the page keeps working (FS-E11)

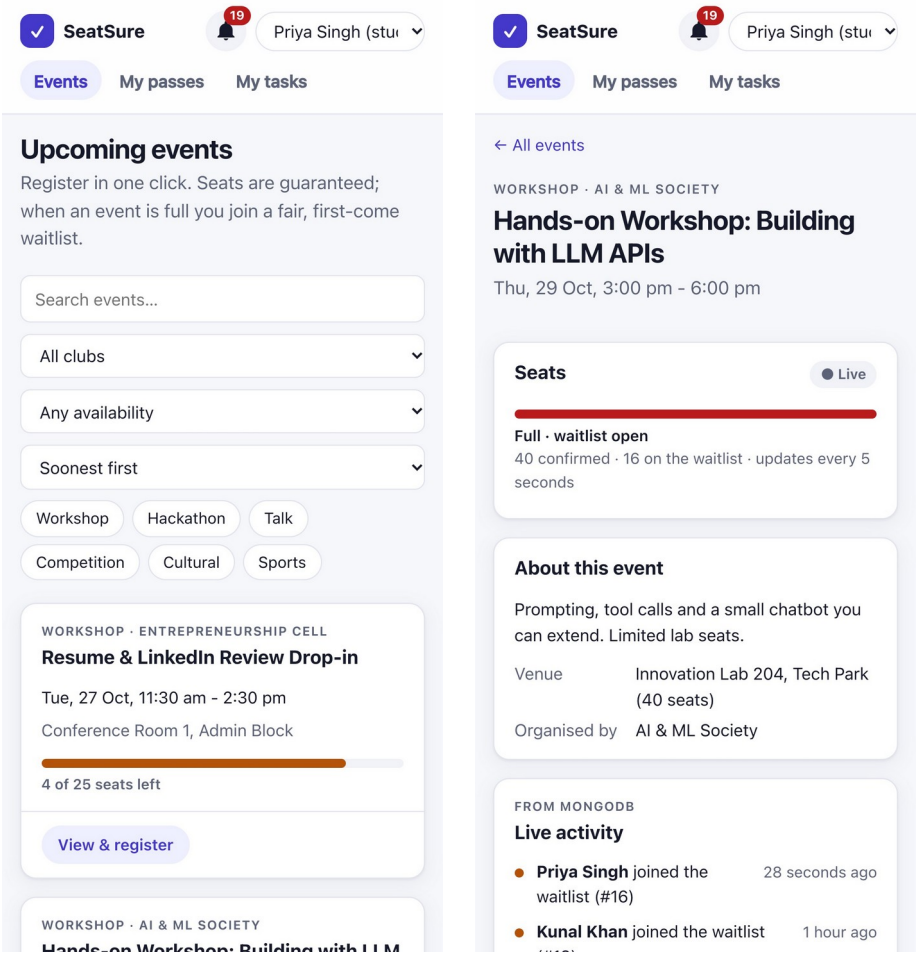


Figure 3.10: Responsive layout on a 390-pixel phone screen: catalogue and event page (FS-E13)

3.4 Live API Test Cases

Besides the automated suites, 21 real HTTP requests were sent to the running API on 27 October 2026 with `npm run test:api`, which records each expected and actual result (raw output in `docs/test-results/api-checks.txt`). API-01 to API-03 are the example test cases from the course guidelines (create a task, an empty title and a missing id); the rest cover every other CRUD operation and the main error cases. **21 of 21 behaved as expected** (Table 3.2).

Table 3.2: Live API test cases: expected and actual results (paths relative to `/api/v1`)

Test	Operation	Request	Expected	Actual	Result
API-01	Create	`POST /tasks {title:"Buy notebook"}`	201 new task	201 id 48, completed false, Location /api/v1/tasks/48	Pass
API-02	Invalid input	`POST /tasks {title:""}`	400 error	400 VALIDATION_ ERROR: Title is required	Pass
API-03	Missing ID	`DELETE /tasks/99`	404 not found	404 TASK_NOT_FOUND	Pass
API-04	Read	`GET /tasks`	200, my tasks incl. "Buy notebook"	200 6 tasks (5 to do, 1 done)	Pass
API-05	Update (toggle)	`PATCH /tasks/48 {completed:true}`	200, done with completedAt	200 completed true, completedAt set	Pass
API-06	Update (edit)	`PATCH /tasks/48 {title:"Buy two notebooks"}`	200, new title	200 "Buy two notebooks"	Pass
API-07	Delete	`DELETE /tasks/48`	204 No Content	204 No Content	Pass
API-08	Read (events)	`GET /events?limit=3`	200, 3 events + nextCursor	200 3 events, nextCursor 4	Pass
API-09	Create	`POST /events {title:"API check: Git workshop", …}`	201 + Location, status DRAFT	201 DRAFT, Location /api/v1/events/32	Pass
API-10	Invalid input	`POST /events {title:"", capacity:0, …}`	400 VALIDATION_ ERROR for title and capacity	400 VALIDATION_ ERROR (title, capacity)	Pass
API-11	Read (one)	`GET /events/32`	200, the new draft	200 "API check: Git workshop" (DRAFT)	Pass
API-12	Update	`PATCH /events/32 {title:"… (updated)"}`	200, new title	200 "API check: Git workshop (updated)"	Pass
API-13	Delete	`DELETE /events/32`	204 No Content	204 No Content	Pass
API-14	Missing ID	`DELETE /events/32 (again)`	404 not found	404 EVENT_NOT_FOUND	Pass
API-15	Update (workflow)	`PATCH /events/33 {status:"PUBLISHED"}`	200, PUBLISHED	200 PUBLISHED	Pass
API-16	Create (registration)	`POST /events/33/registrations {answers:{}}`	201, CONFIRMED	201 CONFIRMED	Pass
API-17	Duplicate	Same request again	409 ALREADY_ REGISTERED	409 ALREADY_ REGISTERED	Pass
API-18	Missing user	Same request without X-User-Id	401	401 UNAUTHENTICATED	Pass
API-19	Delete (cancel)	`DELETE /registrations/2517`	200, cancelled	200 previous status CONFIRMED	Pass
API-20	Missing ID	`DELETE /registrations/99999`	404 not found	404 REGISTRATION_ NOT_FOUND	Pass
API-21	Unknown route	`GET /does-not-exist`	404 ROUTE_NOT_ FOUND	404 ROUTE_NOT_FOUND	Pass

3.5 Discussion

The results show that the client meets its objectives. Three observations stand out.

First, **freshness is cheap when rendering is controlled.** Polling keeps the seat count on an open event page within 5 seconds of the truth, at the cost of one small request per viewer every 5 seconds. Because structural sharing keeps unchanged objects identical and the cards are memoised, those polls cost almost nothing in rendering, as FS-T27 shows.

Second, **optimism is used only where the outcome is known.** Cancelling is optimistic, because the user already knows the result and a rollback is easy to explain. Registering is not, because only the server, holding a database lock, can decide between a seat and the waitlist. Showing a seat that might turn into a waitlist place a moment later would be worse than waiting a fraction of a second for the answer.

Third, **the limits of mocked tests are covered elsewhere.** The client tests mock the network, so on their own they cannot detect a mismatch between what the client sends and what the server expects. That risk is covered by the Back-End integration tests, which exercise the same endpoints, and by the screenshots of the real system. It could be removed completely with browser end-to-end tests (Chapter IV).

Chapter IV – Conclusion

This report presented the Full Stack Web Development part of SeatSure, a campus event-registration system in which seats cannot be overbooked and cancelled seats pass automatically to the next student on the waitlist. The React client applies the concepts of all three modules of the course. Module 1 shaped the structure of the interface: Context and custom hooks for shared state, React Hook Form for validated and dynamically generated forms, `useReducer` for related filters, measured memoisation, lazy-loaded nested routes and error boundaries. Module 2 connected it to the API: one Axios instance with interceptors, TanStack Query caching with per-data freshness, selective invalidation and an optimistic cancellation with rollback. Module 3 carried the work down to the database: filters and sorting executed by Prisma, offset pagination for organiser tables, cursor pagination with infinite scroll for the catalogue, and polling for live seat counts.

All 32 automated tests pass, and screenshots of the running system show every feature working against the real API and databases. The most important lesson of the project is the division of responsibility: the client makes the experience fast and forgiving, but it never makes decisions that belong to the server. Seat allocation stays inside a database transaction, and the client's job is to show that decision quickly, clearly and correctly.

4.1 Limitations

- Identity is a demo user switcher (`X-User-Id`), not real authentication, so anyone can act as any user. This is intentional, because authentication is taught in Module 4.
- Live updates use polling. With thousands of simultaneous viewers, pushing changes from the server would be cheaper than polling.
- There are no browser-level end-to-end tests. The screenshots were captured by a script, but their content is not checked automatically.

4.2 Future Scope

- Add login with sessions or JSON Web Tokens (Module 4) and remove the user switcher.
- Replace polling on the event page with server-sent events, so that seat counts update the moment they change.

- Add browser end-to-end tests (for example with Playwright) for the full register → cancel → promote journey.
- Package the application with Docker and add a continuous-integration pipeline (Module 5).

References

1. Kalvium, *Full Stack Web Development* livebook, Semester 3: Module 1 (Git and Advanced React), Module 2 (Frontend–Backend Integration), Module 3 (Full Stack with Database). Accessed 26 October 2026.
2. React documentation: `createContext`, `memo`, `useMemo`, `useCallback`, `useReducer`, `lazy` and `Suspense`. https://react.dev/reference/react
3. React Router v6 documentation: layout routes, index routes and `<Outlet/>`. https://reactrouter.com/6.30.0
4. TanStack Query v5 documentation: query keys, infinite queries, query invalidation and optimistic updates. https://tanstack.com/query/v5/docs
5. React Hook Form documentation: `useForm`, `useFieldArray` and `setError`. https://react-hook-form.com/docs
6. Axios documentation: instances and interceptors. https://axios-http.com/docs/interceptors
7. react-error-boundary documentation (README). https://github.com/bvaughn/react-error-boundary
8. Vite documentation: environment variables and modes; build and code splitting. https://vite.dev/guide/
9. Prisma ORM documentation: pagination (offset and cursor) and the error reference (P2002, P2025). https://www.prisma.io/docs
10. MDN Web Docs: Intersection Observer API; Cross-Origin Resource Sharing (CORS). https://developer.mozilla.org
11. SeatSure project documentation: `README.md` (traceability matrix), `DECISIONS.md` and `docs/testing/test-cases-fullstack.md`.

Appendix A – Test Cases

Automated tests: `npm run test:frontend` (Vitest, React Testing Library, jsdom), 27 October 2026. Result: **32 / 32 passed**. Raw output: `docs/test-results/fullstack-tests.txt`. Module numbers refer to the Full Stack livebook.

Table A.1: Full Stack test cases (automated)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
FS-T01	M1.11 `useReducer`	Category chip toggles	`toggleCategory TALK` twice	`'TALK'`, then `''`	As expected	Pass
FS-T02	M1.11 `useReducer`	Reset all filters; unknown action	search, club, availability, then reset	3 active, then the initial state; unknown action throws	As expected	Pass
FS-T03	M2.5 query params	Only non-empty filters are sent	`category: 'TALK'`	`{category:'TALK', sort:'soonest'}`	As expected	Pass
FS-T04	M1.5 custom hook (debounce)	Typing "h", "ha", "hack"	299 ms, then 300 ms	Old value, then `'hack'`	As expected	Pass
FS-T05	M2.2 Axios instance + interceptor	Request as a demo user	user 11	`X-User-Id: 11` header	As expected	Pass
FS-T06	M2.10 error normalisation	422 with field errors	`answers.tshirt` error	`ApiError {status 422, code, requestId, fieldErrors}`	As expected	Pass
FS-T07	M2.10 401 handling	401 response	—	Unauthorised handler called (redirect to the user chooser)	As expected	Pass
FS-T08	M2.10 network failure	Server down	Adapter throws	`ApiError {status: 0, code: NETWORK_ERROR}`	As expected	Pass
FS-T09	M1.6 React Hook Form	Fields built from JSONB questions; empty submit	2 questions, no answers	Both fields rendered; "required" error; API **not** called	As expected	Pass
FS-T10	M2.9 RHF + mutation, server errors	Server rejects an answer	422 `answers.tshirt`	Message shown under that field; payload `{tshirt:'M', laptop:'true'}`	As expected	Pass
FS-T11	M2.6 mutation success	Full event → waitlist	Outcome WAITLISTED #3	"you are #3 on the waitlist"	As expected	Pass
FS-T12	M1.4 Context (no user)	Anonymous visitor	—	"Choose user" link to `/choose-user`	As expected	Pass
FS-T13	M3.5 nested include data	My passes	registration → event → club / venue	Title, club and venue rendered	As expected	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
FS-T14	**M2.8 optimistic update + rollback**	Cancel, then the server refuses	Cancel resolves 409 later	Moves to "Cancelled" **before** the response, then rolls back to "Upcoming" with an error toast	As expected	Pass
FS-T15	M2.7 selective invalidation	Cancel succeeds	200 with promotion	Toast shown; "my passes" query re-fetched (2 calls)	As expected	Pass
FS-T16	**M3.9 cursor + `useInfiniteQuery`, M3.11 infinite scroll**	Sentinel scrolls into view	Page 1 `nextCursor: 2`	Second request uses cursor 2; 3 events; "end" message	As expected	Pass
FS-T17	M2.5 filter → query key	Click "Talk" chip	—	New request `{category:'TALK', sort:'soonest'}`, chip pressed	As expected	Pass
FS-T18	M1.5 debounce + M2.5	Type "git" in search	3 keystrokes	Exactly 1 request with `q: 'git'`	As expected	Pass
FS-T19	M1.6 cross-field validation	End before start; capacity 100 in a 40-seat venue	Form values	Both errors shown, `create` **not** called	As expected	Pass
FS-T20	M1.6 `useFieldArray`	Add 2 questions, switch to choice, remove 1	Clicks	2 rows → options input appears → 1 row	As expected	Pass
FS-T21	M2.9 + payload shaping	Valid form + 1 question; server 422 on capacity	—	Clean payload (ids from labels); server message under Capacity	As expected	Pass
FS-T22	**M1.9 error boundary**	Chart crash, then fix	"Simulate a chart crash"	Fallback shown, rest of the dashboard still visible; chart returns after the fix	As expected	Pass
FS-T23	M1.9 contained failure (degraded data)	MongoDB degraded response	`degraded: true`	Warning shown; PostgreSQL reports still rendered	As expected	Pass
FS-T24	**M1.8 lazy routes + M1.10 nested layout routes**	Open `/organiser`	Memory router	App layout first, then lazy organiser layout + index route; bell shows 2 unread	As expected	Pass
FS-T25	M1.10 index redirect	Open `/`	—	Redirect to `/events` (catalogue heading)	As expected	Pass
FS-T26	M1.10 catch-all route	Unknown path	`/nowhere-at-all`	Lazy "Page not found" inside the layout	As expected	Pass
FS-T27	**M1.7 `React.memo` (measure first)**	List re-renders after a poll, then one event changes	3 cards; same objects; then event 2 changes	No card re-renders for unchanged data; only card 2 re-renders	Render count 3 → 3 → 4	Pass
FS-T28	M2.3 query, M3.5 rendering	My tasks list	2 tasks (1 done)	Both rendered; tabs show To do 1, Done 1	As expected	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
FS-T29	M1.6 RHF validation, M2.6 POST mutation	Add a task: empty, then valid	Click Add with no title; then "Buy notebook"	"Title is required", API **not** called; then `create({title:'Buy notebook', notes:'', dueDate:null})` and the list refetched	As expected	Pass
FS-T30	**M2.8 optimistic update + rollback**	Tick a task done; server refuses	Click the checkbox; PATCH rejects 500	Checked **before** the response, unchecked again after the error	As expected	Pass
FS-T31	M2.7 DELETE mutation	Delete a task	Click Delete	`remove(1)`; task disappears after the refetch; "Task deleted" toast	As expected	Pass
FS-T32	M2.9 server errors on fields	Edit a task; server answers 400	New title; field error for `title`	The server's message appears under the Title field of the edit form	As expected	Pass

Table A.2: End-to-end evidence from the running application (screenshots in `docs/screenshots/`)

ID	Concept	What the screenshot shows	File
FS-E01	M3.7/M3.9 catalogue	Catalogue with live seat meters	`01-catalogue.png`
FS-E02	M2.5 + M1.11	Workshop + "Seats available" filters applied	`02-catalogue-filtered.png`
FS-E03	M1.6 + M3.5	Event page with the JSONB question form and venue details	`03-event-detail-open.png`
FS-E04	M2.6 + M3.12	Joining a full event: waitlist #16 toast, live MongoDB feed	`05-waitlisted-result.png`
FS-E05	M3.5	My passes (nested data)	`06-my-passes.png`
FS-E06	M3.12 polling	Notifications delivered through the outbox	`07-notifications.png`
FS-E07	M1.10 + M3.8	Organiser dashboard with offset pagination	`08-organiser-dashboard.png`
FS-E08	M1.6	Event form validation messages	`09-event-form-validation.png`
FS-E09	M3.7 + M3.8	Attendees filtered by a JSONB answer	`11-attendees-jsonb-filter.png`
FS-E10	M2.7	Check-in mutation with selective invalidation	`10c-checked-in.png`
FS-E11	M1.9	Error boundary fallback around one chart	`13-error-boundary.png`
FS-E12	Reliability	MongoDB outage: notifications degraded, registration page still works	`14-…png`, `15-…png`
FS-E13	Responsive UI	390 px phone layout	`16-mobile-catalogue.png`, `17-mobile-event-detail.png`
FS-E14	Tasks: Read	My tasks list with To do / Done counts	`22-tasks-list.png`
FS-E15	Tasks: validation	Empty title rejected before sending ("Title is required")	`23-task-validation.png`
FS-E16	Tasks: Create	"Buy notebook" added with notes and a due date	`24-task-created.png`

ID	Concept	What the screenshot shows	File
FS-E17	Tasks: Update	A task ticked done and another renamed ("Changes saved")	`25-task-updated.png`
FS-E18	Tasks: Delete	Task removed ("Task deleted")	`26-task-deleted.png`
FS-E19	Events: Create / Update / Delete	Organiser creates, edits and deletes a draft; a student cancels a registration	`18-…png` to `21-…png`

Appendix B – Concept Traceability

Every concept is tagged in the code as `// [FULLSTACK Mx.y: Concept]`. This table, taken from the project README, links each concept to the files that implement it, a way to demonstrate it, and the tests that prove it. Test IDs starting with BE- or DB- are server and database tests; they are listed in `docs/testing/` and in the Back-End and Advanced Database reports.

Table B.1: Traceability matrix: concept → code → test (Full Stack Web Development)

Module	Concept	Files / functions	How to demo	Test IDs
M1.4	Context API	`features/users/CurrentUserContext.jsx` (`CurrentUserProvider`, `useCurrentUser`); `shared/components/Toast.jsx` (`ToastProvider`)	Switch the user in the header: header, passes, organiser tab and bell all change	FS-T05, FS-T12
M1.5	Custom hooks	`shared/hooks/useDebouncedValue.js`, `useIntersectionObserver.js`; `features/*/hooks/*`	Type in search: one request after typing stops	FS-T04, FS-T18
M1.6	React Hook Form	`features/events/RegistrationPanel.jsx` (fields from JSONB questions); `features/organiser/EventFormPage.jsx` (`useFieldArray`, cross-field `validate`)	Submit an empty registration / an event ending before it starts	FS-T09, FS-T19, FS-T20
M1.7	`React.memo` / `useMemo` / `useCallback`	`features/events/EventCard.jsx` (`memo`); `EventsPage.jsx` (`useMemo` params, `useCallback` loadMore)	Profiler: polling doesn't re-render unchanged cards	FS-T27
M1.8	Code splitting (`React.lazy` + `Suspense`)	`app/router.jsx` (every page lazy); `layouts/AppLayout.jsx` (`<Suspense>`)	Network tab: `OrganiserLayout-*.js` loads only when opening Organiser	FS-T24
M1.9	Error boundaries	`layouts/AppLayout.jsx` (resets on navigation); `features/organiser/AnalyticsPage.jsx` (`Panel` + `QueryErrorResetBoundary`)	Analytics → "Simulate a chart crash"	FS-T22
M1.10	Nested / layout routes	`app/router.jsx`; `layouts/AppLayout.jsx`, `features/organiser/OrganiserLayout.jsx` (`<Outlet/>`)	Organiser tabs keep the shared chrome	FS-T24, FS-T25, FS-T26
M1.11	`useReducer`	`features/events/state/filtersReducer.js`; `EventsPage.jsx`	Category chips / "Clear filters"	FS-T01–T03
M1.12	Compound components / slots	`shared/components/Card.jsx` (`Card.Header/Body/Footer`, `title`/`actions` slots) used by every card	Same Card in catalogue, passes, panels	(all UI tests)
M1.13	Feature-based architecture	`frontend/src/features/*` + barrels, `shared/`, `services/`	Folder tour	—
M2.1	CORS, env, both servers	`backend/src/app.js` (`cors` allowlist); `.env.example`, `frontend/.env.example`; root `npm run dev` (`concurrently`)	`npm run dev`: every API call from :5173 to :4000 is cross-origin	— (no automated CORS test)
M2.2	Axios instance + `VITE_API_URL`	`services/apiClient.js` (`apiClient`, `unwrap`)	—	FS-T05

Module	Concept	Files / functions	How to demo	Test IDs
M2.3–2.4	TanStack Query: QueryClient, caching, `staleTime`	`app/queryClient.js`; `features/events/hooks/useEvents.js`	"Updating…" pill while refetching in the background	FS-T16
M2.5	Filters → query params	`EventFilters.jsx` → `useEventsInfinite(params)` → `eventsApi.list`	Change filters: new request, new cache entry	FS-T17, FS-T18
M2.6–2.7	Mutations + selective invalidation	`useRegister` (`useEvents.js`); `useCreateEvent`/`useUpdateEvent`/`useCheckIn` (`useOrganiser.js`)	Register, then the seat meter and passes refresh	FS-T11, FS-T15
M2.8	Optimistic updates + rollback	`features/registrations/hooks/useMyRegistrations.js` (`useCancelRegistration`)	Cancel a pass: it moves instantly	FS-T14
M2.9	RHF + mutation + server errors	`RegistrationPanel.jsx` `onSubmit` (`setError`); `EventFormPage.jsx`; `shared/lib/format.js` (`toFormField`)	Server 422 shows under the field	FS-T10, FS-T21
M2.10	Axios interceptors, retry config	`services/apiClient.js` (`toApiError`, 401 → `/choose-user`); `app/queryClient.js` (no 4xx retries)	Clear the user, then try to register	FS-T06–T08
M2.11	Custom API hooks + key factory	`shared/lib/queryKeys.js`; `features/*/hooks/*`	—	(all hook tests)
M3.1–3.2	Prisma connected to Express	`db/src/client.ts` (`prisma`); `backend/src/container.js`	—	BE-I01
M3.3	`include` / `select` / `_count`	`db/src/repositories/eventRepository.ts` (`cardSelect`, `findDetail`)	Event page waitlist count	DB-Q02
M3.4	Prisma errors P2002 / P2025 in the UI	`db/src/errors.ts` → `backend/src/middleware/errorHandler.js` → toast	Register twice: "already registered"	BE-U28, BE-I14
M3.5	Nested include data in React	`features/registrations/PassCard.jsx`	My passes (event → club → venue)	FS-T13
M3.7	Filtering & sorting end-to-end	`EventFilters` → `events.validators.js` → `eventRepository.listPublished`	Sort "Most popular"	FS-T17, BE-I05, DB-Q02
M3.8	Offset pagination	`OrganiserHome.jsx`, `AttendeesPage.jsx`, `shared/components/Pagination.jsx`	Attendees page 2	BE-I17, DB-Q09
M3.9	Cursor pagination + `useInfiniteQuery`	`useEventsInfinite` (`useEvents.js`); `eventRepository.listPublished` (`cursor`, `take: limit+1`)	Scroll the catalogue	FS-T16, BE-I04, DB-Q01
M3.11	Infinite scroll (IntersectionObserver)	`useIntersectionObserver.js` + sentinel in `EventsPage.jsx`	Scroll to the bottom	FS-T16
M3.12	Polling, `refetchOnWindowFocus`, `staleTime`	`useEvent` (5 s), `useNotifications` (15 s), catalogue (30 s)	Two windows: seats update live	FS-E06
M1.6, M2.6–2.9	Tasks CRUD in the UI: form validation, mutations, optimistic tick, server errors on fields	`features/tasks/TasksPage.jsx`, `TaskItem.jsx`, `hooks/useTasks.js`; `services/tasksApi.js`	My tasks: add, tick done, edit, delete	FS-T28–T32